1 Group 2

A good report, with detailed analysis. Overall readability could have been improved upon in some sections.

1.1 Q1

The legibility of many of the figures needed improvement; their size was small and the choice of horizontal scaling meant that it was difficult to pick out key aspects of the results demonstrated. The solid blocks demonstrating overall trends over long time periods in particular added very little. The interpretation was in general good, but there was a key misconception that increasing the time horizon reduced the numerical errors. With no change in the step size, there would be no change in the numerical precision; instead it was the transients which were responsible for the observation of different system response in the short- and long-term.

1.2 Q2

The derivation of the global truncation error was a very nice addition. When experimentally determining that the error was of $\mathcal{O}(h^2)$, you really needed to mention what you plotted for a general "$\mathcal{O}(h^2)$" curve and the reasoning behind concluding that your results matched this (particularly as you didn't include this line in your submitted code). In addition, it is not clear that either this or the Ralston curve are parallel to the others. As in the previous section, your plots over long time periods were not very legible - if you wanted to show the trend you could have plotted the envelope of the response. The decaying errors resulted from the transients; it is only when the steady-state is reached that each error is symmetric about 0 and so the maximum absolute error is at its least. You could have mentioned the surprising result that the use of $h = 1.5e^{-6}$ resulted in errors of $\mathcal{O}(10^{-6})$ (due to the large constant C in $e = Ch^2$).

1.3 Q3

Very good analysis, particularly with the use of Laplace Transforms and a Bode plot. Discussion of the band-pass filter properties of the circuit were good, although there were some errors in the calculation of the frequency range of interest.

1.4 Q4

Very good analysis across a wide range of intial and boundary conditions, particularly with reference to the physical interpretation of the effects of the boundary conditions. The missing fucntion *GetTriangle* meant that it was more difficult to reproduce your results. The bonus was done well.

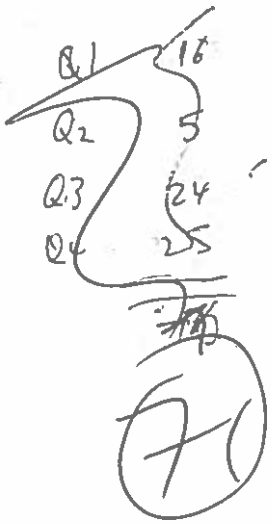
Grade: A

Mathematics Coursework

Numerical Analysis

Second Year

2



Project group:

Alistair Wallace

Piers de Basto

Usman Ahmed

Wenjia Luo

Jonathan Midgen

1 RL circuit

A high-pass filter takes an input signal Vin (for instance, an audio signal) and let only the high- frequency components pass. Usually, high-pass filters are made with capacitors instead of inductors due to the fact that capacitors can be more easily manufactured and are generally physically smaller. However, the example of RL circuit we are going to consider is still very important. In fact, RL circuits are usually used in the modelling of real physical systems such as DC motors.

Consider the RL circuit as depicted in Figure 1. For the circuit, we can write the following equations:

$$Vl(t) + Vr(t) = Vin(t)$$

$$L\frac{dil(t)}{dt} + Ril(t) = Vin(t)$$

The state is iL(t). The input is Vin(t). The output is Vout = Vin(t) − RiL(t). Assume that the initial current through the inductor at time t = 0 is iL(0) = 0 A. We want to model a DC motor with inertia 250 Nm/s2 and maximum torque gain 50 mN m A−1. The corresponding values for the inductance and resistance are:

$$R = 0.5\Omega \quad L = 1.5\text{mH}.$$

Exercise 1. Take three second-order Runge Kutta methods: Heun's and Midpoint, covered in lectures and Ralston's, which you will need to find in the literature. For each method, write two matlab files, so for Ralston's method, there will be (i) a matlab function called ralston.m, implementing the Ralston method to model the RL-circuit, with first line:

function [t,vout]. = ralston(?,?,?,...)

The arguments for the function ralston.m need to be carefully chosen and one will need to be a function; another argument needs to be the final time tf . The function should solve the ODE for any values of R, L, tf and iL(0), not just those given above. The other file should be a script called ralston script.m, in which you call ralston.m to model the circuit for different types of input, and plot the output. When you vary the system parameters and input function, give some thought about choosing tf carefully to allow the features of the output to be seen in the plots, Simulate the system and obtain plots for the following different input signals and observe how the amplitude of the output signal changes:

1. step signal with amplitude V ̄in = 5.5 V;

2. impulsive signal and decay:

$$V_{in} = \bar{V}_{in}\exp\left\{-\frac{t^2}{\tau}\right\} \qquad V_{in} = \bar{V}_{in}\exp\left\{-\frac{t}{\tau}\right\}$$

with $\bar{V}_{in} = 3.5\,\text{V}$ and $\tau = 160\,(\mu\text{s})^2$, resp. $\tau = 160\,\mu\text{s}$.

3. sine, square, and sawtooth waves with amplitude V in = 4.5V and different periods T = 160 s, T = 20 s, T = 450 s, T = 1000 s . For square and sawtooth wave, try Matlab help: "square-", resp. "sawtooth wave".

For each plot, briefly comment on what you see in terms of the circuit and what you know of its behaviour.

$$Vl(t) + Vr(t) = Vin(t)$$

$$L\frac{dil(t)}{dt} + Ril(t) = Vin(t)$$

good to number equations
wrong without subscripts
(not Vx C)

Ex 1 - General Method Explanation

From the two equations above we have a differential equation in il(t). In all three functions an approximate solution to this differential equation in il(t) is found, and then this is used to work out a solution for vout. So the main function which is passed to the numerical method functions is a differential equation in il(t).

Heun's Method

One of the numerical methods used to solve differential equations is called Heun's method. Euler's method uses the tangent of the function and a user-determined step size to determine the next point. Heun's method is different because it predicts the next state by using Euler's method and then takes the average of the gradient with the previous calculated slope:

Euler's Method: $i_{k+1} = i_k + h'f(t_k , i_k)$ ✓

Heun's Method: $i_{k+1} = i_k + 0.5(h'f(t_k , i_k) + h'f(t_{k+1} , i_k + h'f(t_k , i_k))$ ✓

This method of predicting the next i value, evaluating the gradient at this predicted point, then from this calculating an average gradient and predicting a new next i value; can be iterated. Iterating in this fashion can reduce the local truncation error, however as it will not always converge to the exact value of i_{i+1} we decided not to iterate in this way. ✓

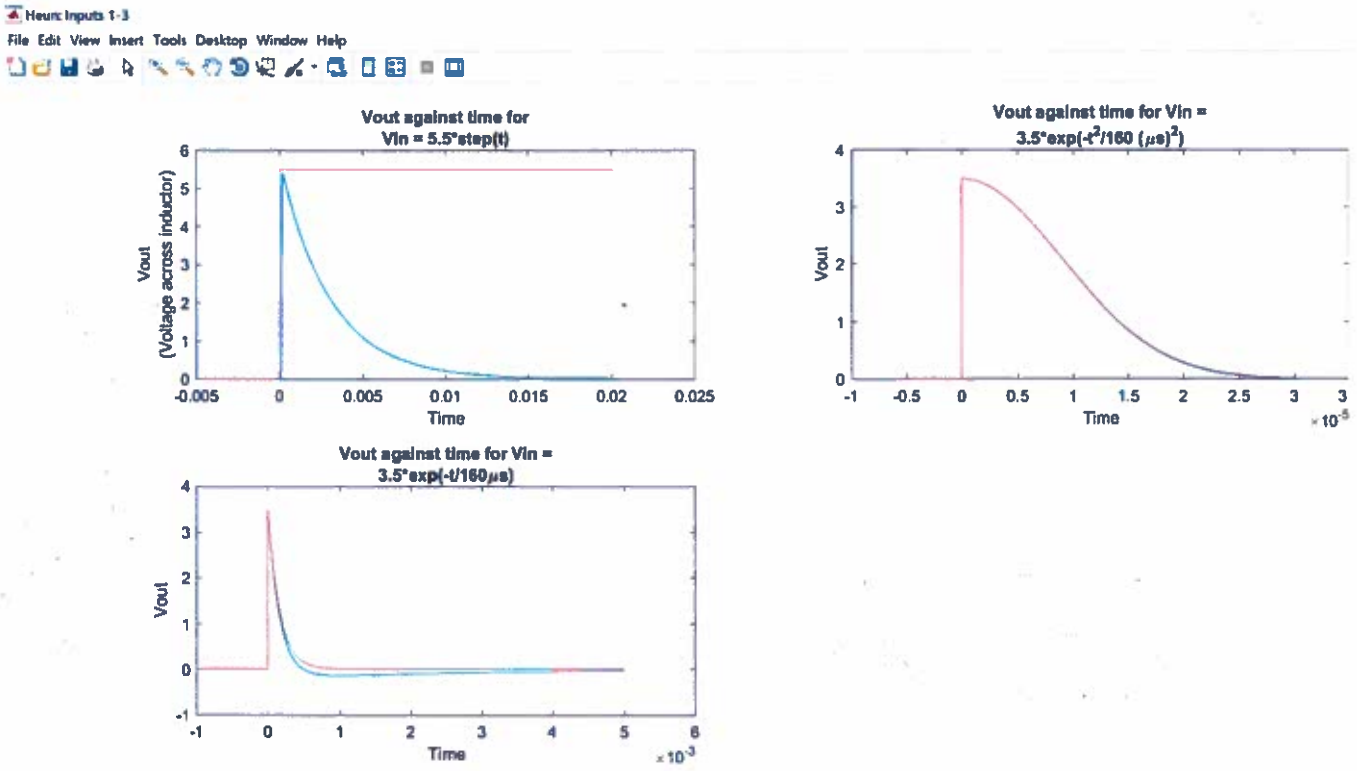
Heun's Method code:

```
function [tout, vout] = heun(dildt, t0, i0, tf, h, Vin, R)
N=round((tf-t0)/h);                 %Nr of steps: (interval size)/(step size)
vout = zeros([N+1 1]); tout = zeros([N+1 1]); %Initialise vectors so we don't
                                    % have to keep changing their size

t = t0; i = i0;                     %Initialise temporary variables: t and i
for j=1:N+1                         %Loop for N+1 steps
    tout(j)=t;                      %Load current t to the output array
    vout(j) = (feval(Vin, t) - R*i);%Load the current vout to the output array
                                    %The above line has been taylored to the
                                    %RL circuit, the rest of the code is general
    grad1 = feval(dildt,t,i);       %Gradient at t
    ip=i + h*grad1;                 %Predictor for next value of i
    grad2 = feval(dildt, t+h, ip);%Gradient at t+h
    igave=0.5*(grad1+grad2);        %Average gradient over [x,x+h]

    i = i+h*igave;                  %New i from euler, using yave as gradient
    t = t + h;                      %Increment t by h
end
end
```

good comments.



Graphs for the Heun Method

Input 1: $Vin = 5.5 * step(t)$

Above are the first three inputs into the Heun function. The first input is a step function with and amplitude of 5.5. As we can see the graph spikes at $t = 0$, and then tails off exponentially. This makes sense, because the RL circuit is a high pass filter, and by the theory of Fourier series, the Heaviside function can be represented by an infinite sum of sine waves. At $t = 0$, there is a discontinuity, and therefore many high frequency terms are needed to represent this change, therefore the high pass filter returns a relatively large value, close to the value of the signal as most of the energy of the signal is at high frequencies at this point. Whereas in steady state when $\pm t \gg 0$, then $\frac{dv}{dt} = 0$, the voltage is DC with a frequency of zero, and so no high frequency terms are needed to represent this function and almost all of the energy of the function is at low frequencies, so the high pass filter returns zero.

Inductor charging

Input 2: $Vin = 3.5 * exp(-\frac{t^2}{160(\mu s)^2})$

For the second function the input is an exponential of a square, the integral of this cannot be found analytically. The numerical method follows this input very closely. I think this is because, if you look, you will see that the timescale is 10^{-5} and so the $\frac{dv}{dt}$ is very large, therefore there are lots of high frequency components and a large proportion of the energy of the signal is at high frequencies, so the high pass filter returns a voltage close to the input.

output

Input 3: $Vin = 3.5 * e^{-\frac{t}{160\mu s}}$

The third input is simply a negative exponential. The output tracks the input closely at the highly discontinuous parts to the signal however it then overshoots and slowly tracks back onto the signal.

why?

Sine Wave Inputs

Square Wave Inputs

Sawtooth Wave Inputs

where are these? later

Midpoint Method

The midpoint method is a predictor-corrector method and it has similarities to the Heun method. It approximates the gradient, dy/dx, at the initial point, where it's denoted as y_i and t_i. Following that it approximates the gradient at the midpoint between t_i and t_i+1, using the first approximation that was done. Both of the approximations are done by using Euler's method. We are then able to find the value of y by multiplying the step size and the second approximation together and then subsequently adding y_i.

If we let K_1 be the first approximation of the derivative in the interval between t_i and t_{i+1}, calculated at the initial points, t_i and y_i. Then,

$K_1 = f(ti , y_i)$

where f(...) is the derivative function.

Then we let K_2 be the second approximation of the derivative in the interval between t_i and t_{i+1}, calculated at the points $t_i + h/2$, where h is the step size, and its corresponding y-value of $yi + k_1/2$. Then,

$K_2 = f(t_i + h/2 , y_i + K_i/2)$

The final y-value at the end of the interval is then,

$y_i + 1 = y_i + hK_2$

The program that implements this function essentially iterates this procedure in defined step sizes from the start of the interval in question, where the initial values are known, to the final t-value specified.

Midpoint Code:

```
function [tout, vout] = midpoint(dildt, t0, i0, tf, h, Vin, R)
N=round((tf-t0)/h);              %Nr of steps: (interval size)/(step size)
vout = zeros([N+1 1]); tout = zeros([N+1 1]); %Initialise vout and tout
                                 %vectors so we don't have to keep creating
                                 %new ones
t = t0; i = i0;                  %Initialise first values
for j=1:N+1                      % loop for N steps
    tout(j) = t;                 %Load current time to time output
    vout(j) = (feval(Vin, t) - R*i);%Load current vout value to vout
                                 %The above line has been specified for the
                                 %RL circuit, the rest of the code is general
    k1 = feval(dildt, t, i);     %Calculate gradient at yi
    k2 = feval(dildt, t + 0.5*h,i +0.5*k1*h);%Estimate gradient at yi + 1/2
    i = i + h*k2;                %Estimate next i
    t = t + h;                   %Increment t by h
end
end
```

Ralston Method:

This is the third Runge - Kutta method for approximating a solution to an input.

The first step is:

$$y_{i+1} = y_i + \frac{1}{4}(k_1 + 3k_2)$$

In which we have:

$$k_1 = h\, f(x_i, y_i)$$

$$k_2 = h\, f(x_i + \tfrac{2h}{3}, y_i + \tfrac{2k_1}{3})$$

$$x_i = x_0 + i\, h$$

By iterating this method, it will give you an approximation for the input signal in the form:

$$y'(x) = f(x, y); \quad y(x_0) = y_0$$

Ralston Code:

```
function [tout, vout] = ralston(dildt, t0, i0, tf, h, Vin, R)
N=round((tf-t0)/h);                 %Nr of steps: (interval size)/(step size) +1
vout = zeros([N+1 1]); tout = zeros([N+1 1]); %Initialise vectors so we don't
                                    %have to keep changing their size

t = t0; i = i0;                     %Initial values for t and i
for j=1:N+1                         %Loop for N+1 steps
    tout(j) = t;                    %Load current t value to tout
    vout(j) = (feval(Vin, t) - R*i);%Load current vout value to vout
                                    %The above line has been specified for th
                                    %the RL Circuit, the rest of the code is general
    k1 = feval(dildt, t, i);        %Calculate k1 for next value
    k2 = feval(dildt, t + (2/3)*h,i +(2/3)*k1*h);%Calculate k2 for next value
    i = i + h*(0.25*k1 + 0.75*k2);%Calculate i for next vout
    t = t + h;                      %Increment t by h
end
end
```

Graphs for Exercise 1

Square wave, Sine Wave and Sawtooth gennerated by the Heun Method

The code that we used to generate these three waves was as follows:

```
%Numerical Methods Coursework - Exercise 1
%Heun Method
%Vin = Vl + Vr
%Vin = L dil/dt + Ril
%Vout = Voltage across the inductor
%Defining Parameters
R = 0.5;        %Resistance
L = 1.5e-3;     %Inductance
i0 = 0;         %Initial Inductor Current

t0 = -0.01;       %Initial Time for test 1
tf = -4*t0;        %Final Time for test 1
h = 0.0001;       %Step Size for test 1
Vin =@(t) (5.5*heaviside(t));
dildt =@(t, il) ((Vin(t) - R*il)/L);   %Vout = L*dil/dt
[tout, vout1] = heun(dildt, t0, i0, tf, h, Vin, R);

figure,
set(gcf,'numbertitle','off','name','Heun: Inputs 1-3')
subplot(2,2,1);
plot(tout, vout1);
title({'Vout against time for'; 'Vin = 5.5*step(t)'});
xlabel('Time');
ylabel({'Vout'; '(Voltage across inductor)'});
grid on;


%-------------------------------------------------------------------
%New Values for second input test
%Vin = 3.5*exp(t^2/160 (?s)^2)
tau = 160e-12;      %Time period
Vinamp = 3.5;
t0 = -0.00002;
tf = 0.0001;        %Final time = 4*time period (Should have made 98% of change)
h = 1e-10;
Vin =@(t) (Vinamp*exp(-t^2/tau))*heaviside(t);   %Set Vin for second test
dildt =@(t, il) ((Vin(t) - R*il)/L);   %Vout = L*dil/dt
[tout2, Vout2] = heun(dildt, t0, i0, tf, h, Vin, R);

subplot(2,2,2);
plot(tout2, Vout2);
title({'Vout against time for Vin ='; '3.5*exp(-t^2/160 (\mus)^2)'});
xlabel('Time');
ylabel('Vout');
grid on;


%-------------------------------------------------------------------
%-------------------------------------------------------------------
%New Values for third input test
%Vin = 3.5*exp(t/160?s)
tau = 160e-6;
Vinamp = 3.5;
t0 = -0.001;
tf = 0.005;         %Final time = 4*time period (Should have made 98% of change)
h = 1e-6;
Vin =@(t) (Vinamp*exp(-t/tau))*heaviside(t);   %Set Vin for second test
dildt =@(t, il) ((Vin(t) - R*il)/L);   %Vout = L*dil/dt
[tout3, Vout3] = heun(dildt, t0, i0, tf, h, Vin, R);

subplot(2,2,3);
plot(tout3, Vout3);
title({'Vout against time for Vin ='; '3.5*exp(-t/160\mus)'});
xlabel('Time');
ylabel('Vout');
grid on;
```

```
%-------------------------------------------------------------
%-------------------------------------------------------------
%New Values for fourth input test - Sine, Square, Sawtooth
%SINE WAVES
%Constant across all sine wave
Vinamp = 4.5;
LSN = 500000;    %Large Scale Width
SSN = 5;         %Small Scale Width
SSAF = 20;      %Small Scale Accuracy Factor (h = 1/SSAF)
LSAF = 4;       %Large Scale Accuracy Factor (h = 1/LSAF)
timeperiods = zeros([4, 1]);
timeperiods(1) = 160e-6;
timeperiods(2) = 20e-6;
timeperiods(3) = 450e-6;
timeperiods(4) = 1000e-6;
NoOfFrequencies = 4;
figure(2)
set(gcf,'numbertitle','off','name','Heun: Sine Waves')
figure(3)
set(gcf,'numbertitle','off','name','Heun: Square Waves')
figure(4)
set(gcf,'numbertitle','off','name','Heun: Sawtooth Waves')

for j= 1:NoOfFrequencies
%-------------------------------------------------------------
```

```
%-------------------------------------------------
%SINE WAVE
%Small Scale
    T = timeperiods(j);     %Time period
    t0 = -T;
    tf = SSN*T;
    h = T/SSAF;
    Vin =@(t) (4.5*sin((2*pi*t)/(T))*heaviside(t));   %Set Vin for fourth test
    dildt =@(t, il) ((Vin(t) - R*il)/L);   %Vout = L*dil/dt
    [tout, Sine] = heun(dildt, t0, i0, tf, h, Vin, R);

    figure(2),
    subplot(2,4,j);
    plot(tout, Sine);
    str = sprintf('Period, T = %g',floor(round(T*1e6)));
    title({str; '(\mus) small scale'});
    xlabel('Time');
    ylabel('Vout');

 %Large scale
    h = T/LSAF;
    t0 = -1;
    tf = T*LSN;
    [tout_large, Sine_large] = heun(dildt, t0, i0, tf, h, Vin, R);

    subplot(2,4,(j + 4));
    plot(tout_large, Sine_large);
    str = sprintf('Period, T = %g',floor(round(T*1e6)));
    title({str; '(\mus) large scale'});
    xlabel('Time');
    ylabel('Vout');



%-------------------------------------------------------------
%SQUARE WAVE
%Small Scale
    t0 = -T;
    tf = SSN*T;
    h = T/SSAF;
    Vin =@(t) (4.5*square((2*pi*t)/(T))*heaviside(t));   %Set Vin for fourth test
    dildt =@(t, il) ((Vin(t) - R*il)/L);   %Vout = L*dil/dt
    [tout, Square] = heun(dildt, t0, i0, tf, h, Vin, R);

    figure(3),
    subplot(2,4,j);
    plot(tout, Square);
    str = sprintf('Period, T = %g',floor(round(T*1e6)));
    title({str; '(\mus) small scale'});
    xlabel('Time');
    ylabel({'Vout'; '(Voltage across inductor)'});

%Large Scale
    h = T/LSAF;
    t0 = -1;
    tf = LSN*T;
    dildt =@(t, il) ((Vin(t) - R*il)/L);   %Vout = L*dil/dt
    [tout_large, Square_large] = heun(dildt, t0, i0, tf, h, Vin, R);

    subplot(2,4,(j + 4));
    plot(tout_large, Square_large);
    str = sprintf('Period, T = %g',floor(round(T*1e6)));
    title({str; '(\mus) large scale'});
    xlabel('Time');
    ylabel('Vout');
```

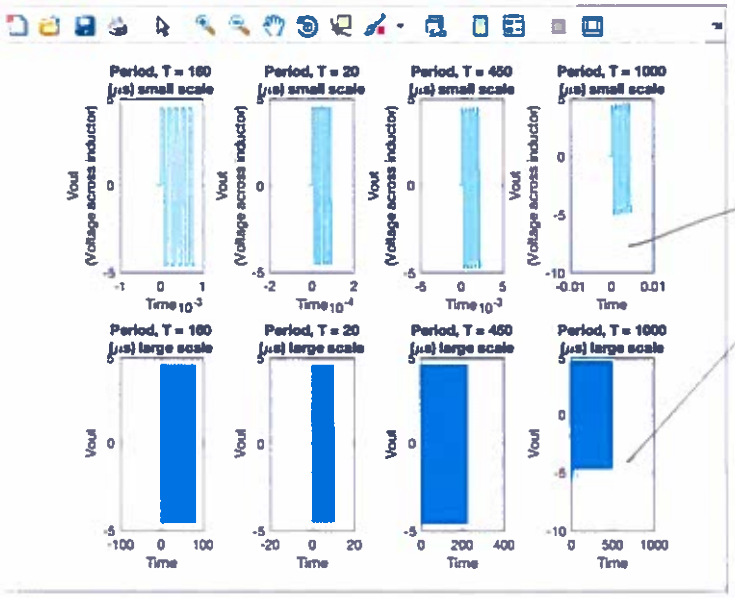
```
%-------------------------------------------------------------------
%SAWTOOTH WAVE
%Small Scale
    t0 = -T;
    tf = SSN*T;
    h = T/SSAF;
    Vin =@(t) (4.5*sawtooth((2*pi*t)/(T))+heaviside(t));   %Set Vin for fourth test
    dildt =@(t, il) ((Vin(t) - R*il)/L);   %Vout = L*dil/dt
    [tout, Sawtooth] = heun(dildt, t0, i0, tf, h, Vin, R);

    figure(4),
    subplot(2,4,j);
    plot(tout, Sawtooth);
    str = sprintf('Period, T = %g',floor(round(T*1e6)));
    title({str; '(\mus) small scale'});
    xlabel('Time');
    ylabel({'Vout'; '(Voltage across inductor)'});

%Large Scale
    h = T/LSAF;
    t0 = -1;
    tf = LSN*T;
    dildt =@(t, il) ((Vin(t) - R*il)/L);   %Vout = L*dil/dt
    [tout_large, Sawtooth_large] = heun(dildt, t0, i0, tf, h, Vin, R);

    subplot(2,4,(j + 4));
    plot(tout_large, Sawtooth_large);
    str = sprintf('Period, T = %g',floor(round(T*1e6)));
    title({str; '(\mus) large scale'});
    xlabel('Time');
    ylabel('Vout');

end
```



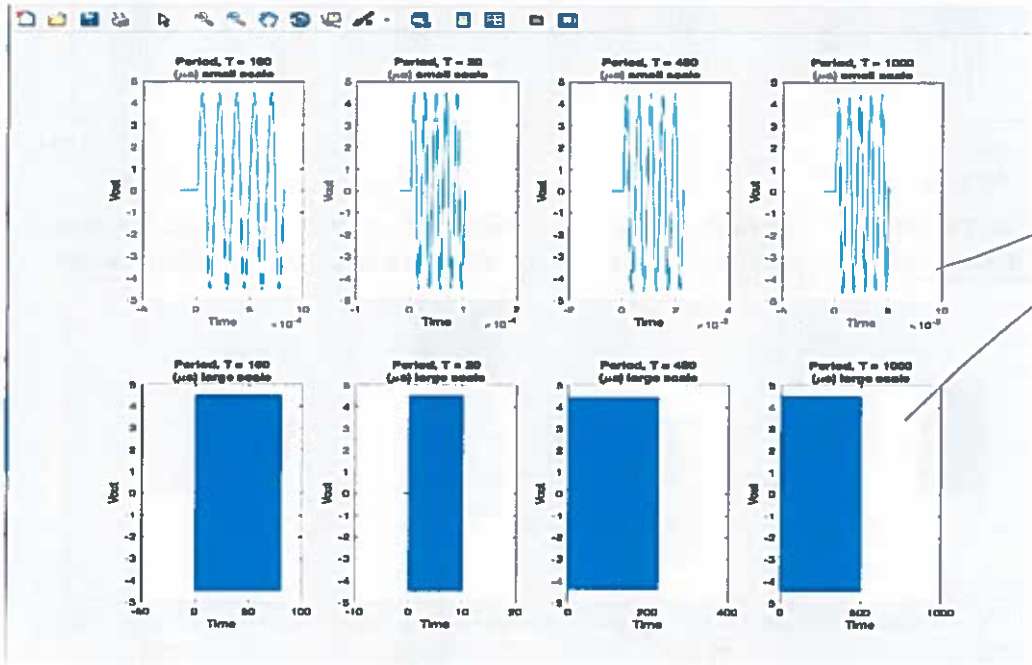
Hard to see

The figure above shows what our output looks like from our square wave input put through the system. What we see from the graphs we get for Vout is that as the frequency of the input square wave is decreased, we see that the square wave we get on Vout gets distorted. This is because the system acts as a high pass filter so the low frequencies will get filtered out. When T = 20μs then that implies that we have a high frequency input so the square wave is made up of more high frequency waves, so less of the input will get filtered out. But as the time period increases we will have a lower frequency square wave input, made up of more waves of lower frequency which means they will get filtered out and we wont see them on the output.

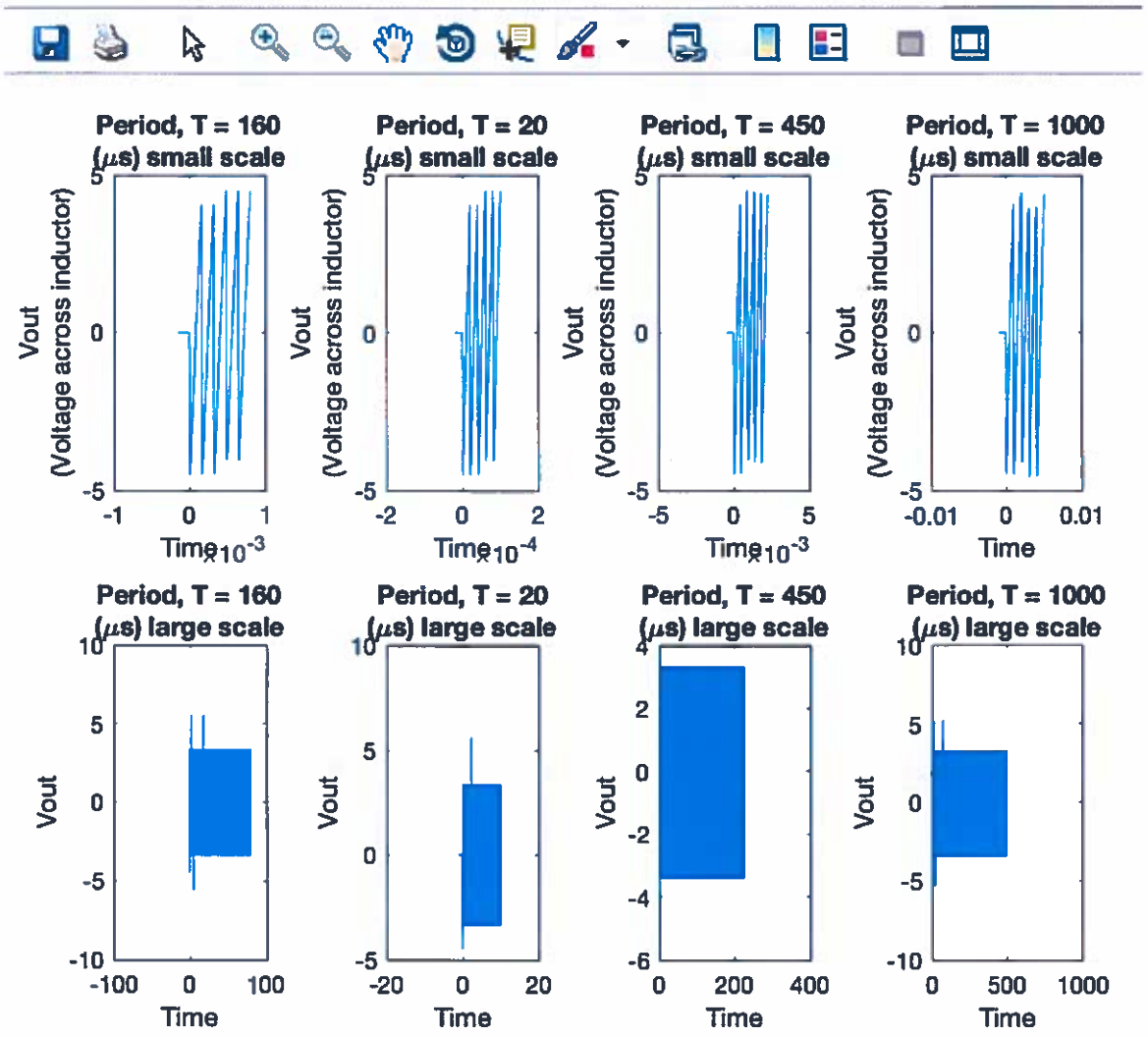
Secondly note how at lower frequencies the top of the square wave becomes sloped, however the discontinuity remains accurate represented. We theorise that this is because, high pass filter does not

attenuate the high frequency terms of the Fourier series which makes up the wave, and so the discontinuity is still accurately represented. However flat sections at the top and bottom of the wave need to be represented by lower frequency and DC terms, which are filtered out by the high pass filter. Because of this tops and bottoms become sloped, and the effect is more pronounced at lower frequency square waves because a higher proportion of the waves' energy is represented by low frequency terms. Therefore, the high pass filter attenuates a larger proportion of the wave and so the wave becomes more disfigured.

When we first observed the square waves over five periods we noticed that the peak voltage varied slightly on the T = 1000μs, so we looked at the amplitude over fifty thousand periods, this showed that the amplitude is relatively constant between different periods.



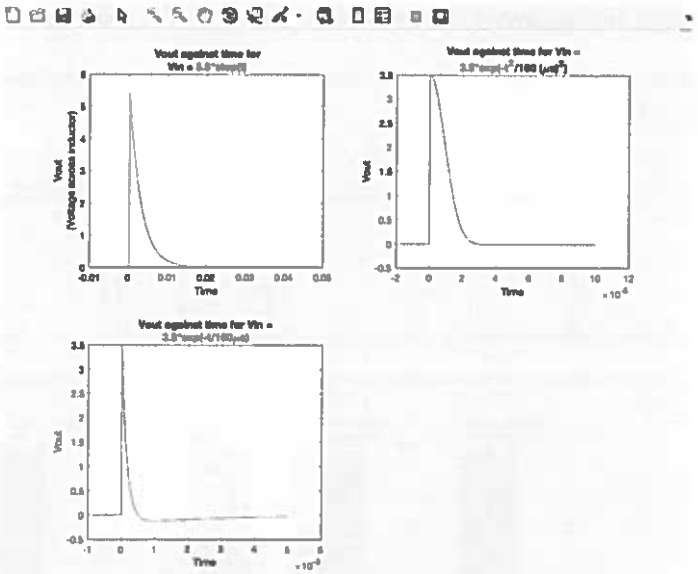
The diagrams above show what we get from the Heun method when we feed a sine wave input into our system. We see that Vout is not distorted very much, and this is because the frequency that we are feeding it is above the cut off frequency of the high pass filter. This means that we are getting the whole input at the output as it is not being attenuated. For sine waves we again observed that the amplitude was relatively constant between oscillations.



The above shows the output we received from a sawtooth inputs at a range of frequencies. We originally noticed that the amplitude of these sawtooth waves was not uniform between waves, so we ran the program over fifty thousand iterations and noticed that the approximation gives weird spikes. This could be due to two factors, either the solution has some amplitude spikes in it, or the numerical method struggles to cope with the approximation over large time periods. We theorised that the second was almost definitely responsible, and having calculated the solution by hand we found it had no obvious causes for these spikes.

we haven't changed the period, just the num length

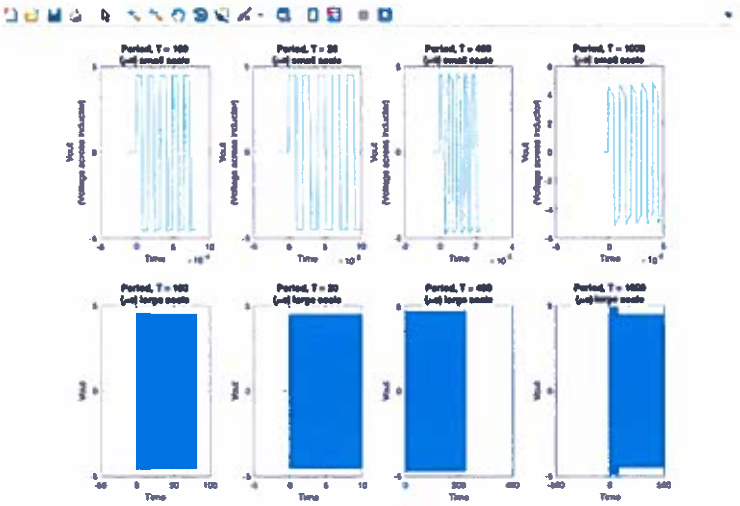
Transients are responsible.



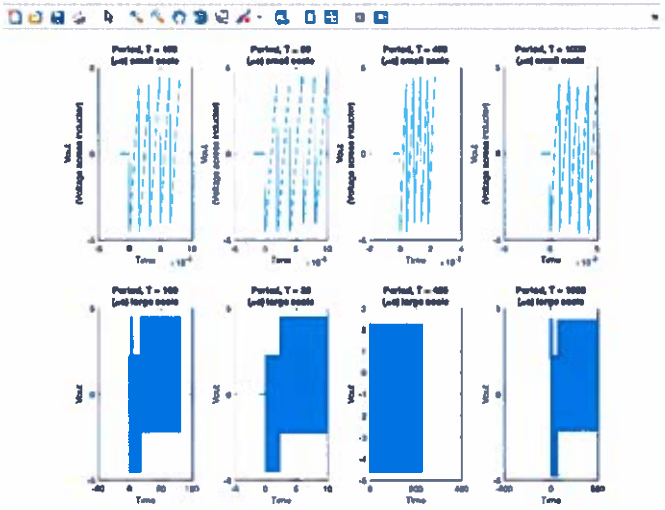
No interpretation

Square wave, Sine Wave and Sawtooth gennerated by the Midpoint Method

We used the same scripts as we did when we are looking at the Heun Method except we changed all the functions from Heun to Midpoint.



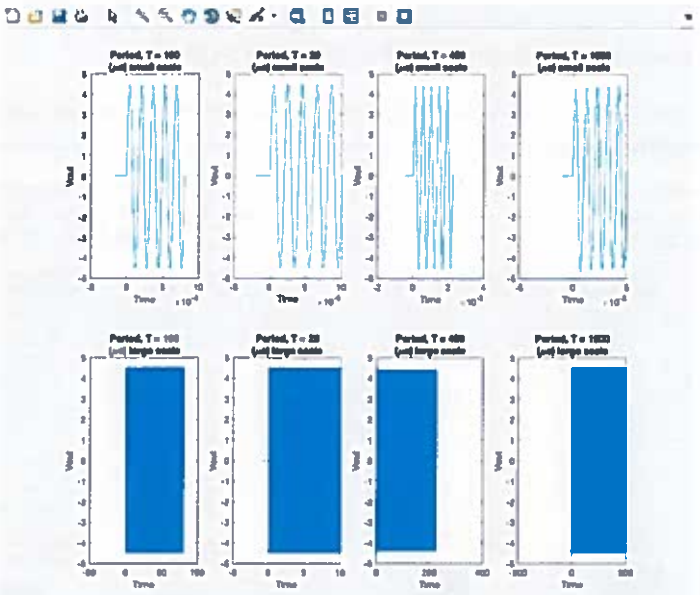
The figure above shows the square wave output that we are getting from the midpoint method. As you can see we are getting very similar outputs as we got when we did the Heun method. This is expected because they system is still acting as a high pass filter. See the square wave output in the Heun section above for a full explanation.



What we can see above in the figure is the sawtooth Vout that we get when we used the midpoint method. We can see that on the small scale the amplitude is changing and we aren't getting a constant amplitude that we expect. The reason for this is because the midpoint method has Local Truncation error dur to the higher order terms in the Taylor Expansion, these terms decay away with time. For more

transients, not error decaying

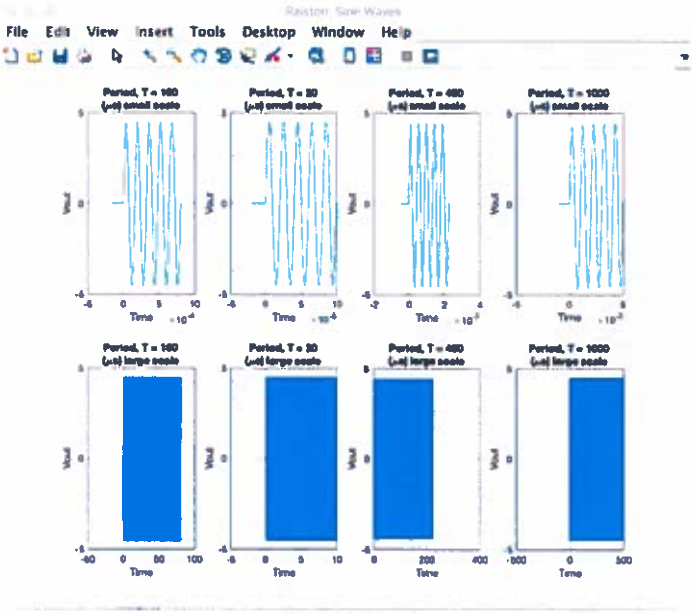
detail on how this effect works see the Error against Time Over a Large Time Period graphs and explanation in Ex2.

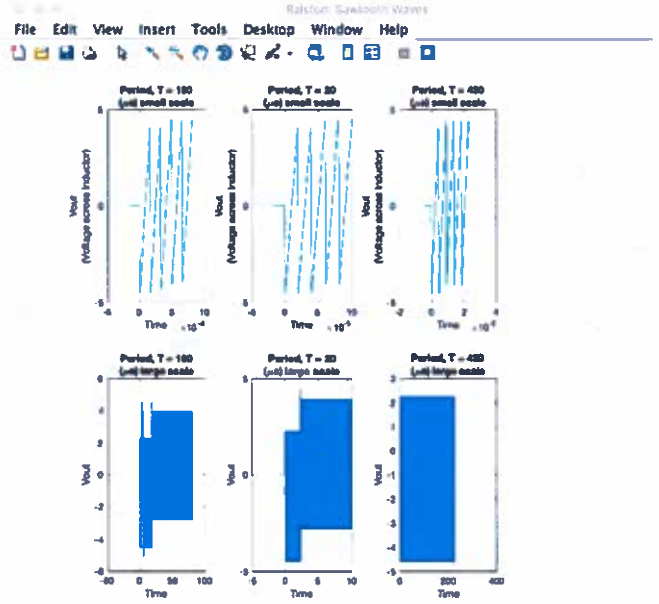


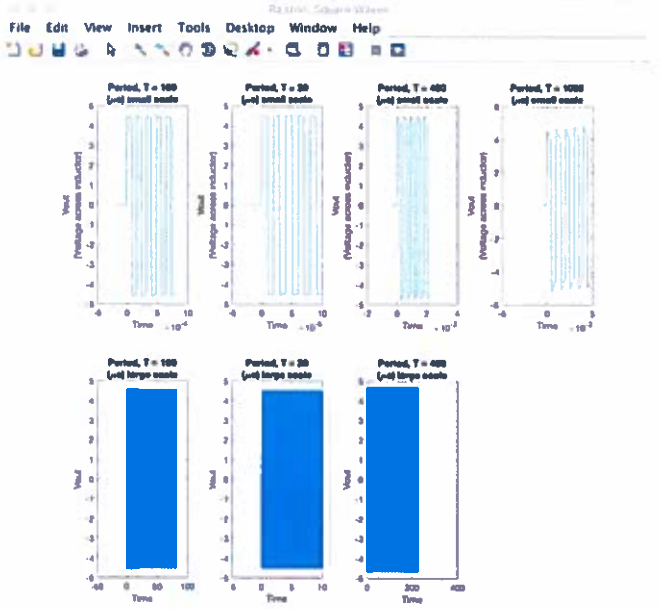
The figure above shows the sine wave output that we get from using the midpoint method, and we you can see, for the same reason as in the Heun method, we are getting a good representation on the output for the sine wave. This is essentially because the input is only made up of a single frequency wave and therefore as long as this frequency is above the cut off frequency for the filter then the signal will be shown clearly on the output. We can also see that the amplitude of all the sine waves has stayed constant which confirms that the frequency that they are at isn't even close to the cut off frequency because if it was then we would expect to see the amplitude start to decrease.

Square wave, Sine Wave and Saw tooth generated by the Ralston Method

We used the same scripts as when we were looking at the Heun Method however we called all the Ralston function instead of the Heun.







What we can see from these graphs of the sine wave, saw tooth wave, and square wave, is that we are getting similar approximations to the other methods and the errors in these for the same reasons. (see explanations for this in the Heun and Midpoint analysis above).

Exercise 2 – Error Analysis of Heun, Midpoint and Ralston

The main advantage of using numerical methods to solve a problem is that you do not need to find the analytical solution, therefore you can solve problems that cannot be solved analytically, such as integrating $e^{x^2}dx$. However, price you pay for this is a loss in accuracy. This error can be reduced by techniques such as repeated iteration and minimising step size, however it can never be completely eliminated.

So the most important analysis of a numerical method is the analysis of its error, both to understand how it can be reduced, and to be able to approximate an error for working out a tolerance when using these methods for solving engineering problems.

Minimising Error:

.

Error Analysis: General

The errors in the three numerical analysis methods that we have used are all going to be the same. This is because they are all second order and because all three methods are derived from the taylor series, and the Taylor series has an error of h^2, where h is the step size.

The theory behind this is:

We know that the error that we are observing is the Global Truncation error, which is upper bounded by the sum of the Local Truncation errors.

$$\boldsymbol{Global\ Truncation} \leq \sum_{1}^{N} \boldsymbol{Local\ Truncation}$$

This is an upper bound as this assumes that all the errors have the same sign and so no errors cancel. This happens using Euler when approximating a curve $f(x)$ where $\frac{d^2f(x)}{df^2} \geq 0\ \forall x$ within the section you are approximating and so $\frac{df(x)}{dx}$ continuously increases over this period.

Therefore when we find the local truncation error we get:

From Heun's method:

$$i_{k+1} = i_k + 0.5(h \times f(t_k , i_k) + h \times f(t_{k+1} , i_k + h \times f(t_k , i_k))$$

When we then expand the last term and substitute that:

$$f(t_k + h, i_k + hf(t_k, i_k)) \approx f(t_{k+1} , i_k + h \times f(t_k , i_k) = f(t_k , i_k) + h \times f_t(t_k , i_k) + h \times f(t_k , i_k) \times f_i(t_k , i_k) + O(h^2)$$

$$i_{k+1} = i_k + h \times f + 0.5h^2 \times (f_t + f_i \times f) + O(h^3)$$

We can also show that this equation has the same first three terms as the Taylor expansion for $i(t_{k+1}+1) = i(t_k + h)$. Therefore, we can conclude that the Local Truncation Error is $O(h^3)$.

Therefore, then we put that into the Global Truncation error equation:

$$Global\ Truncation\ Error \leq N * Local\ Truncation\ Error$$

$$Local\ Truncation\ Error \leq O(h^3)$$

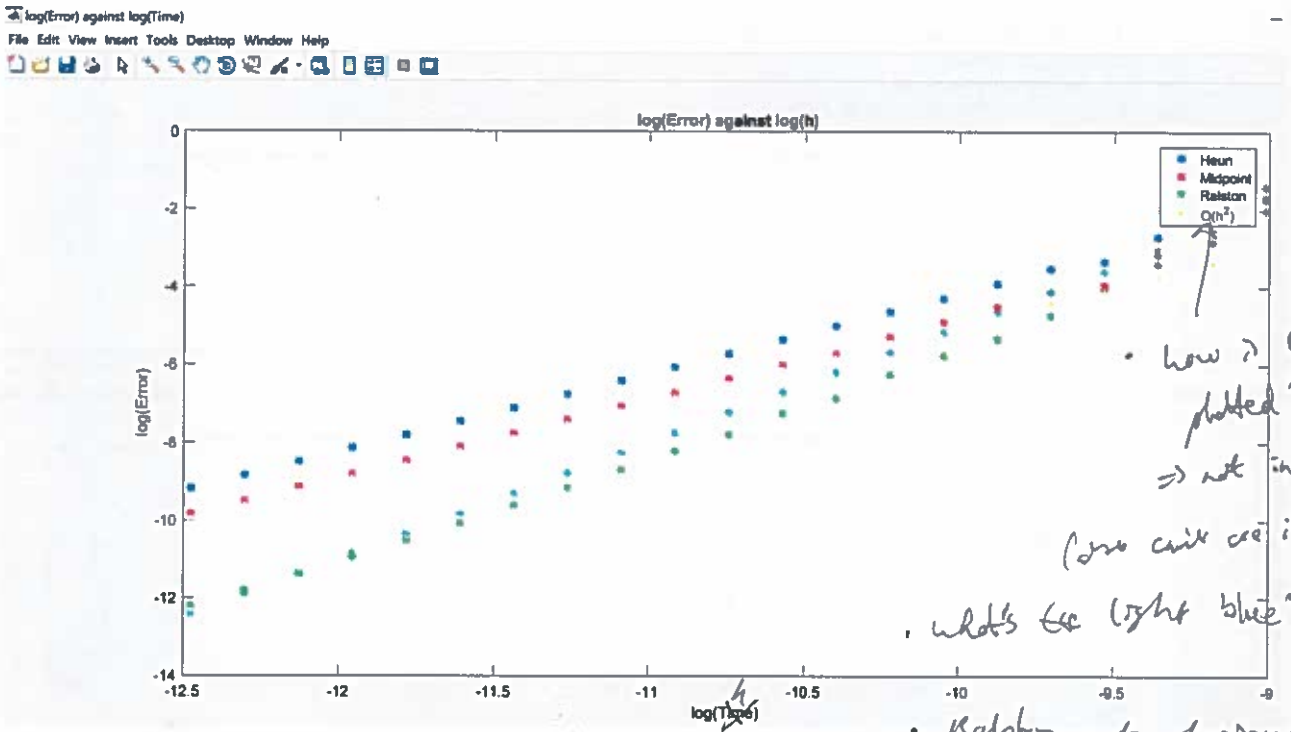
$$n = \frac{t_f - t_0}{h}$$

$$Global\ Truncation \leq \sum_{1}^{N} O(h^3) = n \cdot O(h^3) = \frac{t_f - t_0}{h} \cdot O(h^3) = O(h^2)$$

Therefore, we expect that for the Heun's Method we will get an error of $O(h^2)$ which is what we get on the graph or the error plotted on a log-log graph with h, we see that the gradient is 2.

We know that all Runge-Kutta Methods have the global error so we can conclude that the error for the midpoint and Ralston method is also going to have an error or $O(h^2)$ which is also what we see from the log-log graph below. You can see this because they all have the same gradient, and that is the same gradient as the $O(h^2)$ plot. The gradient is only the same at small log(h) values. This is because at relatively large values of h the $O(h^n)$ terms where $n \geq 2$ are still relatively significant.

This comes from the Taylor Expansion:



• how is this plotted?
⇒ not in script
(you can see it)

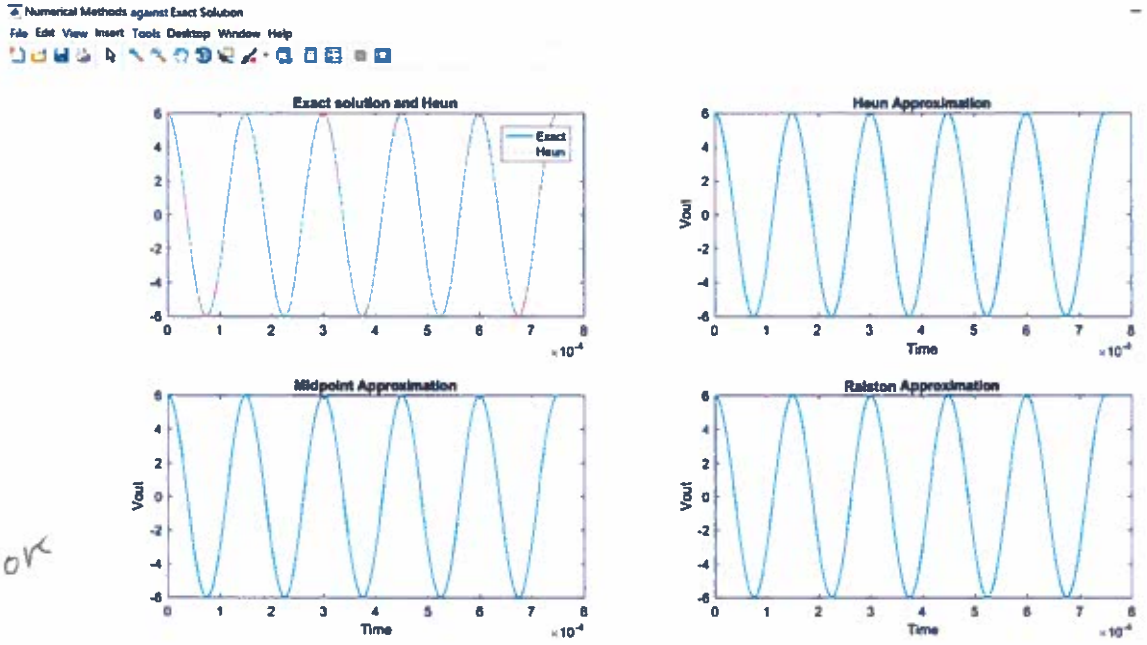
• what's the light blue?

• Ralston at obviously grad of 2

Direct comparison

Below we can see the graphs showing the approximations that the Heun, Midpoint and Ralston method give for the specified input. These graphs show that the approximation is broadly accurate and gives a good general approximation of the output. To further show this we also plotted the Heun approximation on the same graph as the exact solution which shows that they overlap almost exactly. This does not

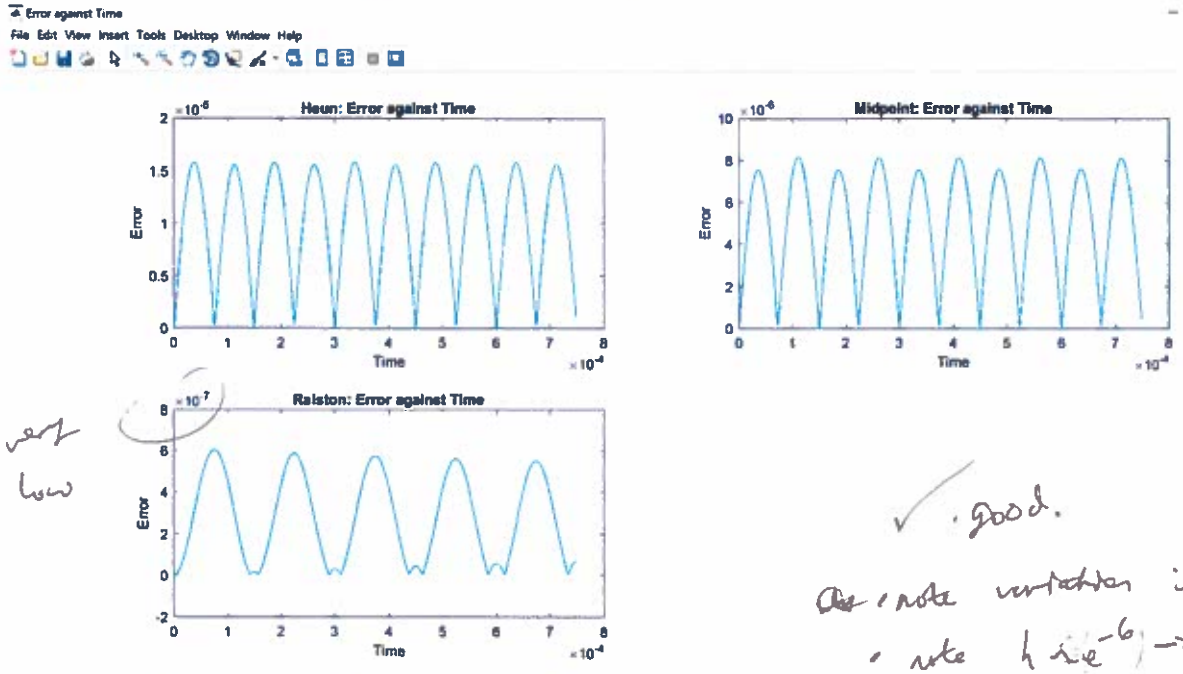
how the error accurately, however it gives a general overview.



ok

Error against time – Short Period

The next graph shows the error for all three methods against time. This is the global error and we can see that it is both periodic and different for each of the three methods.

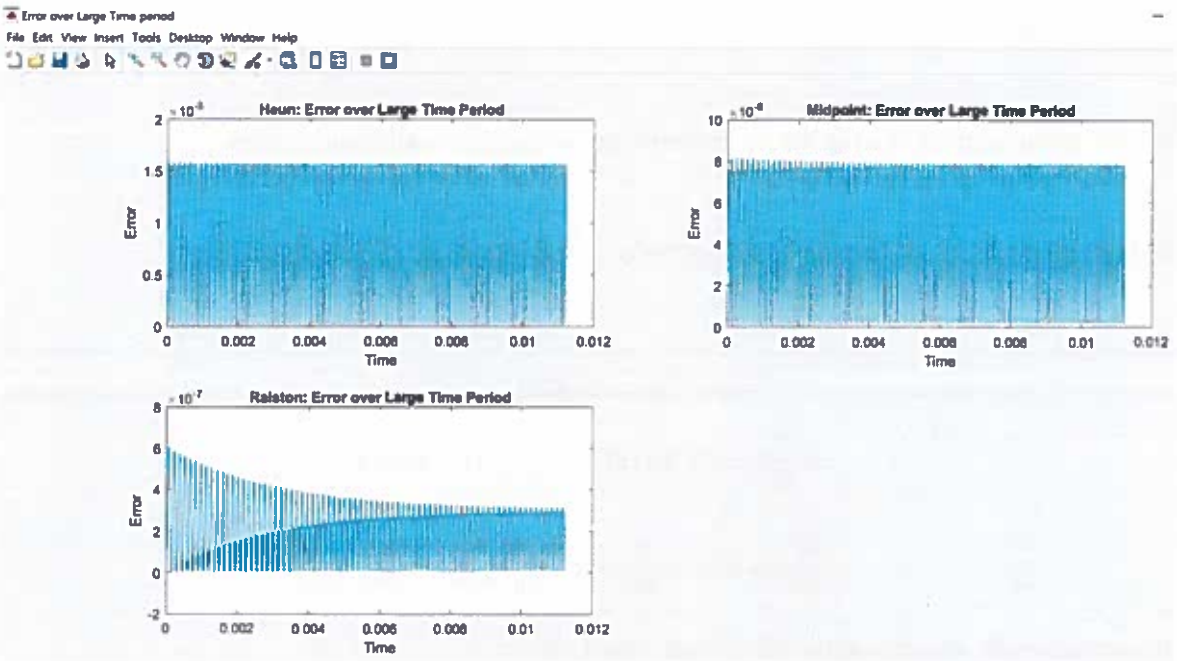


very low

✓ good.

Also note variation in errors
• note $h = 1e^{-6}$ → error ~ e^{-7}

The team noticed that the amplitudes of errors of the approximations was not constant, and so decided to observe the error over a larger time period, the results of which can be seen in the graph below.



Heun Error Analysis:

Don't put title when thanks.

Midpoint Error Analysis:

It can be seen in Error against time graph that the error does not have constant amplitudes, however it is shown in the Large Time Period Erro graph that the global error converges to a constant waveform over a larger number of waveforms. This is due to the higher order terms in the Taylor expansion decaying.

Raltson Error Analysis:

It can be clearly seen in the Error against Time graph that the amplitude of the error is not constant between oscillations. Alternate periods have vastly different amplitudes. This effect is then shown more clearly in the Large Time Period Graph. It can be seen that the Global Error converges to a constant value.

Another interesting observation is that the local truncation error of the Ralston Error is significantly smaller.

Exercise 3

The RLC circuit consists of a resistor, an inductor, and a capacitor connected in series.

According to KVL, the RLC second order ODE for q can be obtained

:

$$V_L(t) + V_R(t) + V_C(t) = V_{in}(t)$$

$$L\frac{d}{dt}i_L(t) + Ri_L(t) + \frac{1}{C}\int_0^t i_L(t) = V_{in}(t)$$

$$L\frac{d^2}{dt^2}q_C(t) + R\frac{d}{dt}q_C(t) + \frac{1}{C}q_C(t) = V_{in}(t)$$

The characteristic equation of the RLC second order ODE is

characteristic eqn
∴ of form $\lambda^2 + 2\lambda + 1 = 0$

$$L\frac{d^2}{dt^2}q_C(t) + R\frac{d}{dt}q_C(t) + \frac{1}{C}q_C(t) = 0$$

The general solution, which is also the zero-input response is

$$q_c(t) = c_1 e^{-233.333t}\sin(601.849t) + c_2 e^{-233.333t}\cos(601.849t)$$

The system is underdamped.

$\zeta = \frac{R}{2}\sqrt{\frac{C}{L}}$ =0.36 is damping ratio. When $0 \leq \zeta < 1$, the system is underdamped, which means the system oscillates with the amplitude gradually decreasing to zero.

$\omega_n = \frac{1}{\sqrt{LC}} = 645.5$ is undamped natural frequency, which is also the resonant frequency of the RLC circuit. The resonant frequency is defined as the frequency at which the impedance of the circuit is at a minimum. Equivalently, it can be defined as the frequency at which the impedance is purely real. This occurs because the impedances of the inductor and capacitor at resonance are equal but of opposite sign and thus cancel out.

Where $\omega_d = \sqrt{1-\zeta^2}\omega_n = 602$ is the damped natural frequency. It is the frequency the circuit will naturally oscillate at if not driven by an external source.

good.

The 3/8 method is a fourth order Runge-Kutta method for approximating the solution of the initial value problem $\frac{dy}{dx} = f(x,y);\ y(x_0) = y_0$ which evaluates the integrand $f(x,y)$ four times per step.

In the RLC circuit, we have $\frac{dx}{dt} = y$ and $\frac{dy}{dt} = \frac{\text{vin}-\left(\frac{1}{C}\right)\text{x}-\text{Ry}}{\text{L}}$ where x is $qC(t)$ and y is $\frac{d}{dt}qC(t) = i_L(t)$. With initial conditions, x(0)=500 nC and y(0)=0A. For general first order ODE's

$$\frac{dx}{dt} = f(t,x,y) = func1$$

$$\frac{dy}{dt} = g(t,x,y) = func2$$

The fourth order Runge-Kutta formula's for a system of second ODE's are:

$$k_1 = hf(t_i, x_i, y_i);\ l_1 = hg(t_i, x_i, y_i);$$

$$k_2 = hf\left(t_i + \frac{h}{3}, x_i + \frac{k_1}{3}, y_i + \frac{l_1}{3}\right);\ l_2 = hg\left(t_i + \frac{h}{3}, x_i + \frac{k_1}{3}, y_i + \frac{l_1}{3}\right);$$

$$k_3 = hf\left(t_i + \frac{2h}{3}, x_i - \frac{k_1}{3} + k_2, y_i - \frac{l_1}{3} + l_2\right);\ l_3 = hg\left(t_i + \frac{2h}{3}, x_i - \frac{k_1}{3} + k_2, y_i - \frac{l_1}{3} + l_2\right);$$

$$k_4 = hf(t_i + h, x_i + k_1 - k_2 + k_3, y_i + l_1 - l_2 + l_3);$$

$$l_4 = hg(t_i + h, x_i + k_1 - k_2 + k_3, y_i + l_1 - l_2 + l_3)$$

and

$$x_{i+1} = x_i + \frac{1}{8}(k_1 + 3k_2 + 3k_3 + k_4);\ y_{i+1} = y_i + \frac{1}{8}(l_1 + 3l_2 + 3l_3 + l_4);$$

We typically need some inputs for the algorithm:

- A range that we want to do the calculations over: $t_0 \le t \le t_f$
- Input voltage
- The step size h
- The total number of steps $N = \frac{t_f - t_0}{h}$ and the number of steps i.

By repeating the calculation for N time steps. The final result should be close to the exact solution of the second order ODE.

For RK4second.m

```
function [x,y] = RK4second(func1, func2, t0, x0, y0, h)



   k1x = feval(func1, t0, x0, y0);      %evaluate func1

   k1y = feval(func2, t0, x0, y0);      %evaluate func2

                                %fourth order runge kutta 3/8 algorithm

   k2x = feval(func1, t0+(h/3), x0+(h/3)*k1x, y0+(h/3)*k1y);

   k2y = feval(func2, t0+(h/3), x0+(h/3)*k1x, y0+(h/3)*k1y);



   k3x = feval(func1, t0+(2*h/3), x0-(h/3)*k1x+h*k2x, y0-(h/3)*k1y+h*k2y);

   k3y = feval(func2, t0+(2*h/3), x0-(h/3)*k1x+h*k2x, y0-(h/3)*k1y+h*k2y);



   k4x = feval(func1, t0+h, x0+h*k1x-h*k2x+h*k3x, y0+h*k1y-h*k2y+h*k3y);

   k4y = feval(func2, t0+h, x0+h*k1x-h*k2x+h*k3x, y0+h*k1y-h*k2y+h*k3y);



   xx = x0 + h*(1/8)*(k1x+3*k2x+3*k3x+k4x);   %next value of x calculated

   yy = y0 + h*(1/8)*(k1y+3*k2y+3*k3y+k4y);   %find next value of y similarly




x=xx; y=yy;                          %return the three arrays
```

For RLC_script.m

```
x0=500/1000000000;                    %x=qC(t), set initial condition of x at t_0
y0=0;                                 %y=x', set initial condition of y at t_0
t0=0;
R=280;                                %Resistance
C=4/1000000;                          %Capacitance
L=0.6;                                %Inductance
%-----------------------step input with amplitude 5V-------------------
h=0.0001;                             % set step size
tf=0.05;                              % stop here
N=round((tf-t0)/h);

vin =@ (t) 5*heaviside(t);            %define input(step input)
func1 =@ (t,x,y)y;                    %x'=y=f_1
func2 =@ (t,x,y)(feval(vin,t)-(1/C)*x-R*y)/L;   %Lx''+Rx'+(1/C)x=vin(t) --> y'=(vin-(1/C)*x-R*y)/L=f_2

xa = zeros(1,N);ya = zeros(1,N);ta = zeros(1,N);    %set up arrays
xa(1)=x0;ya(1)=y0;ta(1)=t0;           %initialize arrays

for j=1:N-1                           %loop for N steps

xi=xa(j);yi=ya(j);ti=ta(j);           %temporary name
[xx,yy]=RK4second(func1, func2, ti, xi, yi, h);%call to RK4second.m
xa(j+1)=xx;                           %update the value into array
ya(j+1)=yy;
ta(j+1)=ti+h;

end

vout=R*ya;                            %vout = R*dqC(t)/dt

figure;
vinn = 5*heaviside(ta);               %set up an array of the input signal
subplot(2,1,1);                       %divide the figre into a 2x1 matrix and put the plot of the input signal the first row
plot(ta,vinn);                        %plot input signal
xlabel('Time/s')                      %label the axis and add title
ylabel('Vin/V')
title('Step signal with amplitude 5V')
subplot(2,1,2);                       %put the plot of output waveform the second row of the matrix
plot(ta,vout);                        %plot the output waveform
xlabel('Time/s')                      %label the axis and add title
ylabel('Vout/V')
title('Output Waveform')
%-------------impulse signal with decay-------------------
h=0.0001;                             % set step size
tf=0.1;                               % stop here
N=round((tf-t0)/h);

vin =@ (t) 5*exp(-(t^2)/3e-06);       %define input(impulse signal)
func1 =@ (t,x,y)y;                    %x'=y=f_1
func2 =@ (t,x,y)(feval(vin,t)-(1/C)*x-R*y)/L;   %Lx''+Rx'+(1/C)x=vin(t) --> y'=(vin-(1/C)*x-R*y)/L=f_2

xa = zeros(1,N);ya = zeros(1,N);ta = zeros(1,N);    %set up arrays
xa(1)=x0;ya(1)=y0;ta(1)=t0;           %initialize arrays

for j=1:N-1                           %loop for N steps

xi=xa(j);yi=ya(j);ti=ta(j);           %temporary name
[xx,yy]=RK4second(func1, func2, ti, xi, yi, h);%call to RK4second.m
xa(j+1)=xx;                           %update the value into array
ya(j+1)=yy;
ta(j+1)=ti+h;

end

vout=R*ya;                            %vout = R*dqC(t)/dt

figure;
vinn = 5*exp(-(ta.^2)/3e-06);         %set up an array of the input signal
subplot(2,1,1);                       %divide the figure into a 2x1 matrix and put the plot of the input signal the first row
plot(ta,vinn);                        %plot input signal
```

```
xlabel('Time/s')                    %label the axis and add title
ylabel('Vin/V')
title('Impulse signal with decay')
subplot(2,1,2);                     %put the plot of output waveform the s econd row of the matrix
plot(ta,vout);                      %plot the output waveform
xlabel('Time/s')                    %label the axis and add title
ylabel('Vout/V')
title('Output Waveform')
%-------------For square wave with frequency 5 HZ----------------

h=0.0001;                           % set step size
tf=1;                               % stop here
N=round((tf-t0)/h);

vin =@ (t) 5*square(2*pi*5*t);              %define input(square signal)
func1 =@ (t,x,y)y;                  %x'=y=f_1
func2 =@ (t,x,y)(feval(vin,t)-(1/C)*x-R*y)/L;   %Lx''+Rx'+(1/C)x=vin(t) --> y'=(vin-(1/C)*x-R*y)/L=f_2

xa = zeros(1,N);ya = zeros(1,N);ta = zeros(1,N);    %set up arrays
xa(1)=x0;ya(1)=y0;ta(1)=t0;                 %initialize arrays

for j=1:N-1                         %loop for N steps

xi=xa(j);yi=ya(j);ti=ta(j);                 %temporary name
[xx,yy]=RK4second(func1, func2, ti, xi, yi, h);%call to RK4second.m
xa(j+1)=xx;                         %update the value into array
ya(j+1)=yy;
ta(j+1)=ti+h;

end

vout=R*ya;                          %vout = R*dqC(t)/dt

figure;
vinn = 5*square(2*pi*5.*ta);            %set up an array of the input signal
subplot(2,1,1);                     %divide the figre into a 2x1 matrix and put the plot of the input signal the first row
plot(ta,vinn);                      %plot input signal
xlabel('Time/s')                    %label the axis and add title
ylabel('Vin/V')
title('square wave with amplitude 5 V and frequency 5HZ')
subplot(2,1,2);                     %put the plot of output waveform the s econd row of the matrix
plot(ta,vout);                      %plot the output waveform
xlabel('Time/s')                    %label the axis and add title
ylabel('Vout/V')
title('Output Waveform')
%----------square wave with amplitude 5 V and frequency f=100HZ----------

h=0.0001;                           % set step size
tf=0.1;                             % stop here

N=round((tf-t0)/h);

vin =@ (t) 5*square(2*pi*100*t);            %define input
func1 =@ (t,x,y)y;                  %x'=y=f_1
func2 =@ (t,x,y)(feval(vin,t)-(1/C)*x-R*y)/L;   %Lx''+Rx'+(1/C)x=vin(t) --> y'=(vin-(1/C)*x-R*y)/L=f_2

xa = zeros(1,N);ya = zeros(1,N);ta = zeros(1,N);    %set up arrays
xa(1)=x0;ya(1)=y0;ta(1)=t0;             %initialize arrays

for j=1:N-1                         %loop for N steps

xi=xa(j);yi=ya(j);ti=ta(j);             %temporary name
[xx,yy]=RK4second(func1, func2, ti, xi, yi, h);%call to RK4second.m
xa(j+1)=xx;                         %update the value into array
ya(j+1)=yy;
ta(j+1)=ti+h;

end

vout=R*ya;                          %vout = R*dqC(t)/dt
```

```
figure;
vinn = 5*square(2*pi*100.*ta);        %set up an array of the input signal
subplot(2,1,1);                       %divide the figure into a 2x1 matrix and put the plot of the input signal the first row
plot(ta,vinn);                        %plot input signal
xlabel('Time/s')                      %label the axis and add title
ylabel('Vin/V')
title('Square wave with amplitude 5V and frequency 100 HZ')
subplot(2,1,2);                       %put the plot of output waveform the se cond row of the matrix
plot(ta,vout);                        %plot the output waveform
xlabel('Time/s')                      %label the axis and add title
ylabel('Vout/V')
title('Output Waveform')
%---------- square wave with amplitude 5 V and frequency f=500HZ----------

h=0.0001;                     % set step size
tf=0.03;                    % stop here

N=round((tf-t0)/h);

vin =@ (t) 5*square(2*pi*500*t);                    %define input
func1 =@ (t,x,y)y;                          %x'=y=f_1
func2 =@ (t,x,y)(feval(vin,t)-(1/C)*x-R*y)/L;  %Lx''+Rx'+(1/C)x=vin(t) --> y'=(vin-(1/C)*x-R*y)/L=f_2

xa = zeros(1,N);ya = zeros(1,N);ta = zeros(1,N);   %set up arrays
xa(1)=x0;ya(1)=y0;ta(1)=t0;                        %initialize arrays

for j=1:N-1                   %loop for N steps

xi=xa(j);yi=ya(j);ti=ta(j);                   %temporary name
[xx,yy]=RK4second(func1, func2, ti, xi, yi, h);  %call to RK4second.m
xa(j+1)=xx;                   %update the value into array
ya(j+1)=yy;
ta(j+1)=ti+h;

end

vout=R*ya;                    %vout = R*dqC(t)/dt

figure;
vinn = 5*square(2*pi*500.*ta);                %set up an array of the input signal
subplot(2,1,1);                       %divide the figure into a 2x1 matrix and put the plot of the input signal the first row
plot(ta,vinn);                        %plot input signal
xlabel('Time/s')                      %label the axis and add title
ylabel('Vin/V')
title('Square wave with amplitude 5V and frequency 500 HZ')
subplot(2,1,2);                       %put the plot of output waveform the se cond row of the matrix
plot(ta,vout);                        %plot the output waveform
xlabel('Time/s')                      %label the axis and add title
ylabel('Vout/V')
title('Output Waveform')
%----------sine wave with amplitude 5 V and at frequency f=5HZ----------

h=0.0001;                     % set step size
tf=1;                       % stop here

N=round((tf-t0)/h);

vin =@ (t) 5*sin(2*pi*5*t);                    %define input
func1 =@ (t,x,y)y;                          %x'=y=f_1
func2 =@ (t,x,y)(feval(vin,t)-(1/C)*x-R*y)/L;  %Lx''+Rx'+(1/C)x=vin(t) --> y'=(vin-(1/C)*x-R*y)/L=f_2

xa = zeros(1,N);ya = zeros(1,N);ta = zeros(1,N);   %set up arrays
xa(1)=x0;ya(1)=y0;ta(1)=t0;                        %initialize arrays

for j=1:N-1                   %loop for N steps

xi=xa(j);yi=ya(j);ti=ta(j);                   %temporary name
[xx,yy]=RK4second(func1, func2, ti, xi, yi, h);%call to RK4second.m
xa(j+1)=xx;                   %update the value into array
```

```
ya(j+1)=yy;
ta(j+1)=ti+h;

end

vout=R*ya;                      %vout = R* dqC(t)/dt

figure;
vinn = 5*sin(2*pi*5.*ta);           %set up an array of the input signal
subplot(2,1,1);                  %divide the figure into a 2x1 matrix and put the plot of the input signal the first row
plot(ta,vinn);                   %plot input signal
xlabel('Time/s')                  %label the axis and add title
ylabel('Vin/V')
title('Sine wave with amplitude 5V and frequency 5 HZ')
subplot(2,1,2);                  %put the plot of output waveform the se cond row of the matrix
plot(ta,vout);                   %plot the output waveform
xlabel('Time/s')                  %label the axis and add title
ylabel('Vout/V')
title('Output Waveform')
%--------sine wave with amplitude 5 V and at frequency f=100HZ----------
h=0.0001;                     % set step size
tf=0.1;                      % stop here

N=round((tf-t0)/h);

vin =@ (t) 5*sin(2*pi*100*t);              %define input
func1 =@ (t,x,y)y;                      %x'=y=f_1
func2 =@ (t,x,y)(feval(vin,t)-(1/C)*x-R*y)/L;   %Lx''+Rx'+(1/C)x=vin(t) --> y'=(vin-(1/C)*x-R*y)/L=f_2

xa = zeros(1,N);ya = zeros(1,N);ta = zeros(1,N);    %set up arrays
xa(1)=x0;ya(1)=y0;ta(1)=t0;               %initialize arrays

for j=1:N-1                         %loop for N steps

xi=xa(j);yi=ya(j);ti=ta(j);               %temporary name
[xx,yy]=RK4second(func1, func2, ti, xi, yi, h);%call to RK4second.m
xa(j+1)=xx;                         %update the value into array
ya(j+1)=yy;
ta(j+1)=ti+h;

end

vout=R*ya;                      %vout = R* dqC(t)/dt

figure;
vinn = 5*sin(2*pi*100.*ta);           %set up an array of the input signal
subplot(2,1,1);                  %divide the figure into a 2x1 matrix and put the plot of the input signal the first row
plot(ta,vinn);                   %plot input signal
xlabel('Time/s')                  %label the axis and add title
ylabel('Vin/V')
title('Sine wave with amplitude 5V and frequency 100 HZ')
subplot(2,1,2);                  %put the plot of output waveform the se cond row of the matrix
plot(ta,vout);                   %plot the output waveform
xlabel('Time/s')                  %label the axis and add title
ylabel('Vout/V')
title('Output Waveform')
%----------sine wave with amplitude 5 V and at frequency f=500HZ--------
h=0.0001;                     % set step size
tf=0.04;                    %stop here

N=round((tf-t0)/h);

vin =@ (t) 5*sin(2*pi*500*t);              %define input
func1 =@ (t,x,y)y;                      %x'=y=f_1
func2 =@ (t,x,y)(feval(vin,t)-(1/C)*x-R*y)/L;   %Lx''+Rx'+(1/C)x=vin(t) --> y'=(vin-(1/C)*x-R*y)/L=f_2

xa = zeros(1,N);ya = zeros(1,N);ta = zeros(1,N);    %set up arrays
xa(1)=x0;ya(1)=y0;ta(1)=t0;               %%initialize arrays

for j=1:N-1                 %loop for N steps
```

```
xi=xa(j);yi=ya(j);ti=ta(j);                %temporary name
[xx,yy]=RK4second(func1, func2, ti, xi, yi, h);%call to RK4second.m
xa(j+1)=xx;                %update the value into array
ya(j+1)=yy;
ta(j+1)=ti+h;

end

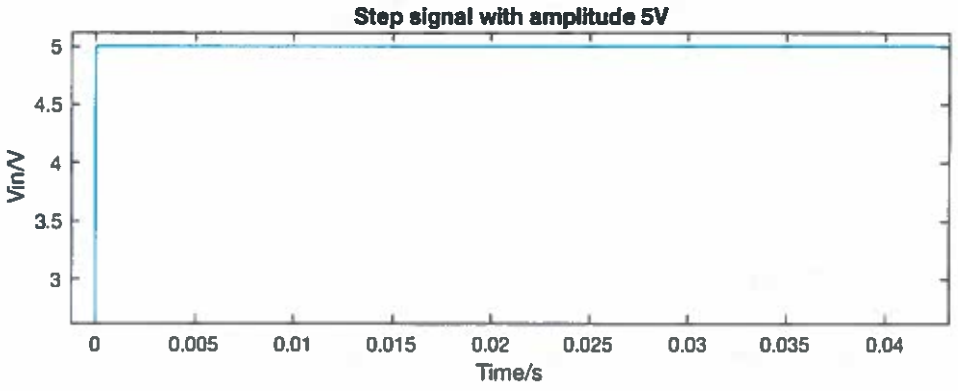
vout=R*ya;                 %vout = R*dqC(t)/dt

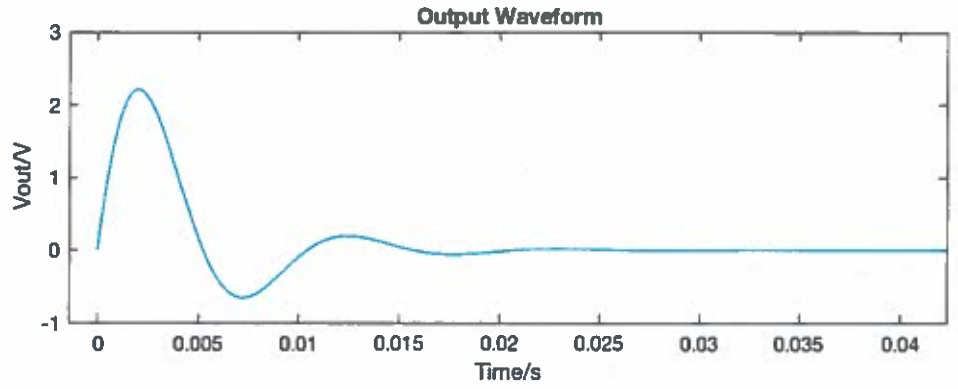
figure;
vinn = 5*sin(2*pi*500.*ta);                %set up an array of the input signal
subplot(2,1,1);                %divide the figure into a 2x1 matrix and put the plot of the input signal the first row
plot(ta,vinn);                 %plot input signal
xlabel('Time/s')               %label the axis and add title
ylabel('Vin/V')
title('Sine wave with amplitude 5V and frequency 500 HZ')
subplot(2,1,2);                %put the plot of output waveform the se cond row of the matrix
plot(ta,vout);                 %plot the output waveform
xlabel('Time/s')               %label the axis and add title
ylabel('Vout/V')
title('Output Waveform')
```

For step signal with amplitude $V_{in} = 5V$

```
vin =@ (t) 5*heaviside(t);          %define input
vinn = 5*heaviside(ta);             %set up an array of the input signal
```

good clarity





could be on same graph.

When connected to a step signal with a voltage of 5V, the capacitor is charged initially. The voltage of the charged capacitor induces a current flow through the inductor which discharges the capacitor. When the capacitor is discharged, the inductor resists the change in the current flow, resulting in the capacitor being charged again with the opposite polarity. The voltage in the capacitor then causes the current flow to stop and then start flowing in an opposite direction, thereby causing oscillations. The oscillation will be diminished by the resistor if there is no driving AC power source in the circuit, which is known as damping.

The differential equation relating the input $V_{in}(t)$ to the output $q_C(t)$ can be written as:

$$M\frac{d^2}{dt^2}q_C(t) + B\frac{d}{dt}q_C(t) + kq_C(t) = V_{in}(t)$$

where

$$M = L, B = R, k = \frac{1}{C}$$

By using Laplace transform, the transfer function $G(s) = \frac{q(s)}{vin(s)}$ can be written as

$$G(s) = \frac{1}{Ms^2 + Bs + k} = K\frac{{\omega_n}^2}{s^2 + 2\zeta\omega_n s + {\omega_n}^2}$$

where

$$K = \frac{1}{k}, \omega_n = \sqrt{\frac{k}{M}}, \zeta = \frac{B}{2\sqrt{kM}}$$

$K = C = 4\times10^{-6}$ is the DC gain.

$V_{in}(t)$ is a step signal with amplitude equalling to 5, so $q(s) = 5\times K\frac{{\omega_n}^2}{s(s^2+2\zeta\omega_n s+{\omega_n}^2)}$

Expanding in partial fractions:

$$q(s) = 5\times K\times(\frac{1}{s} - \frac{s + \zeta\omega_n}{(s + \zeta\omega_n)^2 + {\omega_d}^2} - \frac{\zeta\omega_n}{(s + \zeta\omega_n)^2 + {\omega_d}^2})$$

Taking the inverse Laplace transform

$$q(t) = 5\times K\times(1 - \frac{1}{\sqrt{1-\zeta^2}}e^{-\zeta\omega_n t}\sin(\sqrt{1-\zeta^2}\omega_n t + \alpha))$$

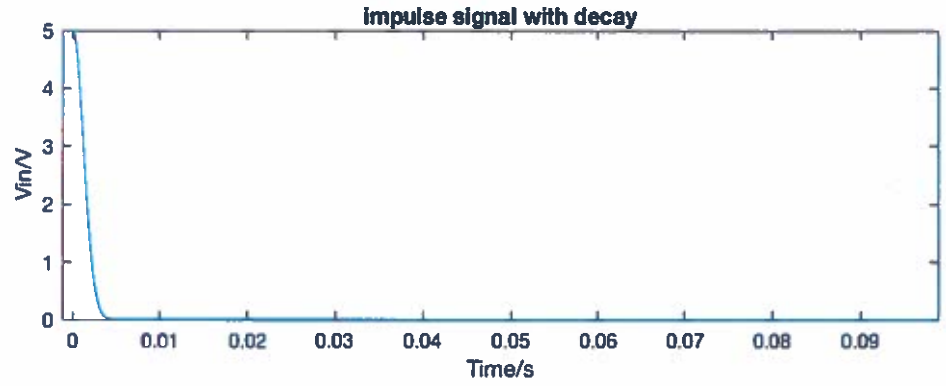
where $\alpha = \tan^{-1}\frac{\sqrt{1-\zeta^2}}{\zeta} = \cos^{-1}\zeta = \sin^{-1}\sqrt{1-\zeta^2}$

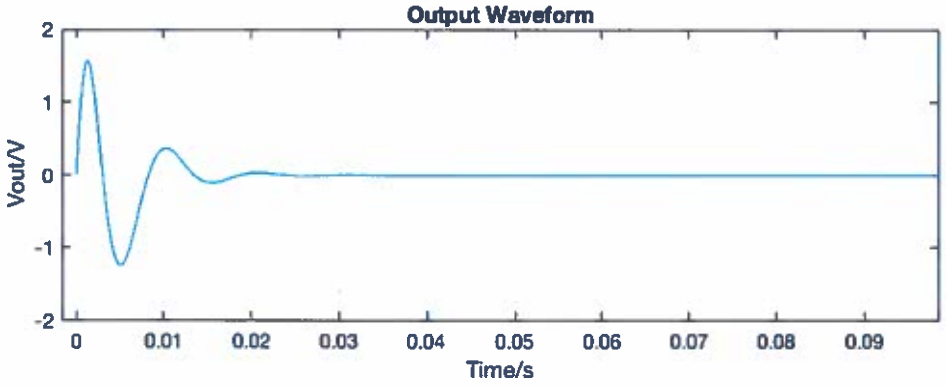
The first term of q(t) is called the steady-state response and 5K is called the steady-state value. The second term is called the transient response and $\tau = \frac{1}{\zeta\omega_n}$ is the time constant of the system.

Differentiate q(t) will get $\frac{dq(t)}{dt}$, which is the current flow through resistor R. To find the maximum value of the output voltage, differentiate $\frac{dq(t)}{dt}$ and equate to 0. Then $t = 1.997\times10^{-3}s$ is the time that the maximum output voltage is obtained, the maximum output voltage is 2.21V.

For an impulse signal with decay $V_{in} = 5e^{-\frac{t^2}{3\times10^{-6}}}$

```
vin =@ (t) 5*exp(-(t^2)/3e-06);          %define input
vinn = 5*exp(-(ta.^2)/3e-06);            %set up an array of the input signal
```

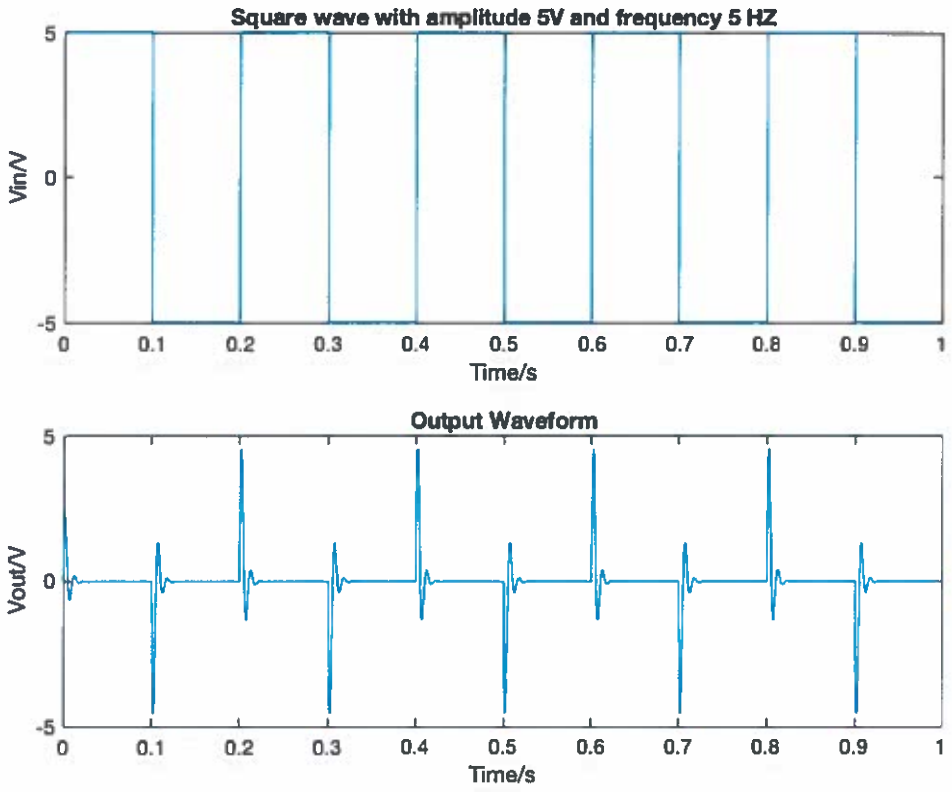




The capacitor cannot be charged by an impulse voltage. The resistor cannot absorb impulse voltage either. As a result, the current in the circuit tends to infinity which is resisted by the inductor. Therefore, the entire impulse voltage passes through the inductor, and results in an increase in the inductor current with rate $\frac{d}{dt}iL(t) = \frac{V}{L} = 8.33\ A/s$. Whilst the impulse signal decays exponentially, the inductor current can only achieve a tiny increase. When the output voltage reaches 1.5665 V, the input source becomes short circuit for $t \geq 0^+$, the circuit becomes a source-free circuit and the response is similarly to zero input response, the voltage oscillates with a decaying amplitude.

For square wave with amplitude 5 V and frequency f=5HZ

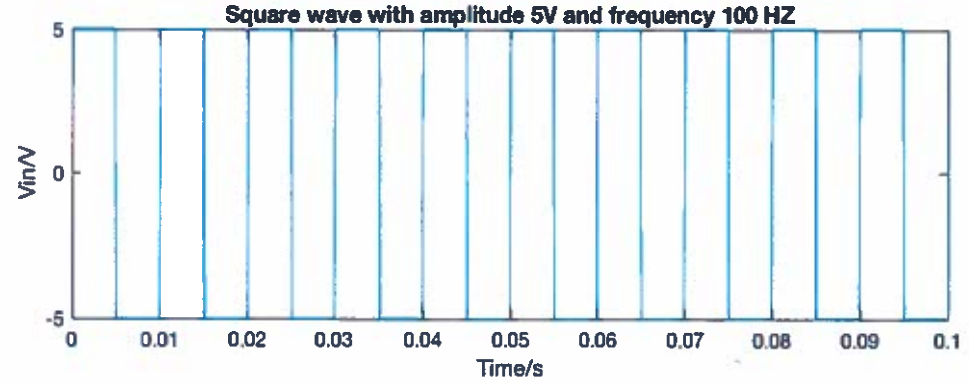
```
vin =@ (t) 5*square(2*pi*5*t);          %define input
vinn = 5*square(2*pi*5.*ta);            %set up an array of the input signal
```

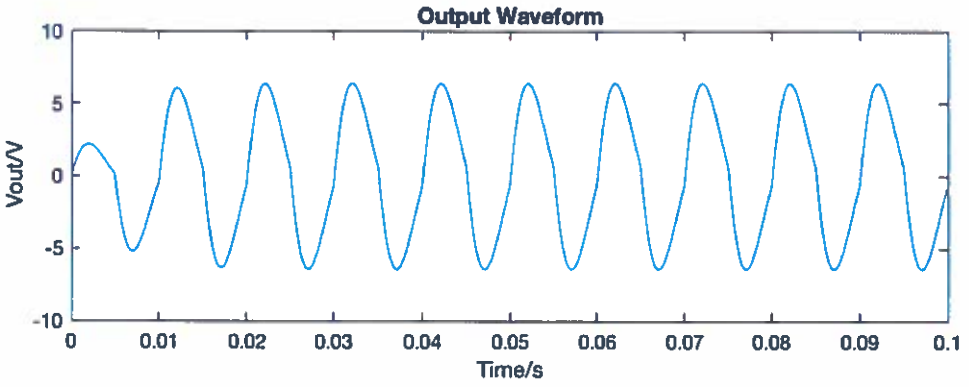


For a square wave with frequency 5 HZ, the output waveform is similar to that of the step response when the amplitude of the input square wave is positive, and gets inverted when the amplitude is negative. The reason why the first pulse is different from others is the transient response caused by a sudden change of input voltage. A transient event is a short-lived burst of energy in a system caused by a sudden change of state.

For square wave with amplitude 5 V and frequency f=100HZ

```
vin =@ (t) 5*square(2*pi*100*t);          %define input
vinn = 5*square(2*pi*100.*ta);            %set up an array of the input signal
```

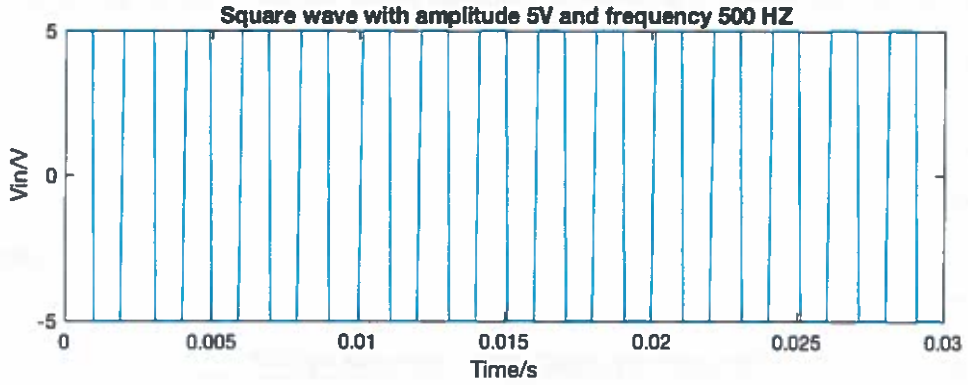


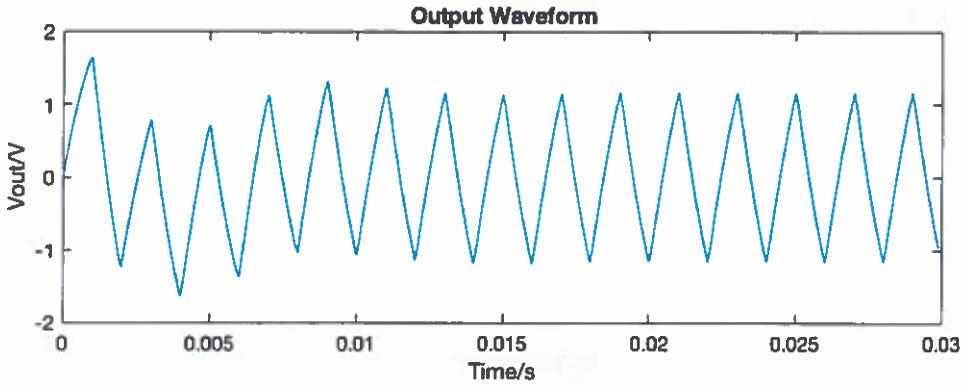


The reason why the first pulse is different from others is the transient response caused by a sudden change of the input voltage. The amplitude of the output waveform for input signal with frequency 100 HZ is the largest due to the impact of the resonant component. Because a square wave can be expressed as the limit case of an infinite series of sinusoidal waves, with fast convergence. 100 HZ is closed to the resonant frequency of this RLC circuit, which is $\frac{645.5}{2\pi} = 102.$

For square wave with amplitude 5 V and frequency f=500HZ

```
vin =@ (t) 5*square(2*pi*500*t);          %define input
vinn = 5*square(2*pi*500.*ta);            %set up an array of the input signal
```





The initial sudden change of the output waveform is caused by transient of the oscillation. The amplitude of the output waveform with the input frequency 500 HZ is the smallest. With a frequency of 500 HZ, the capacitor does not have enough time to be charged and the induced current is small.

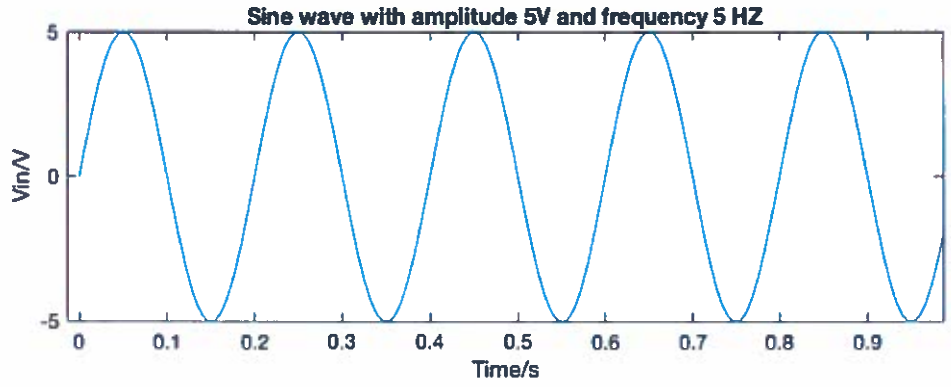
For sine wave with amplitude 5 V and different frequencies f=5HZ, f=110HZ, f=500HZ, a sinusoidal driving force is applied to the system.

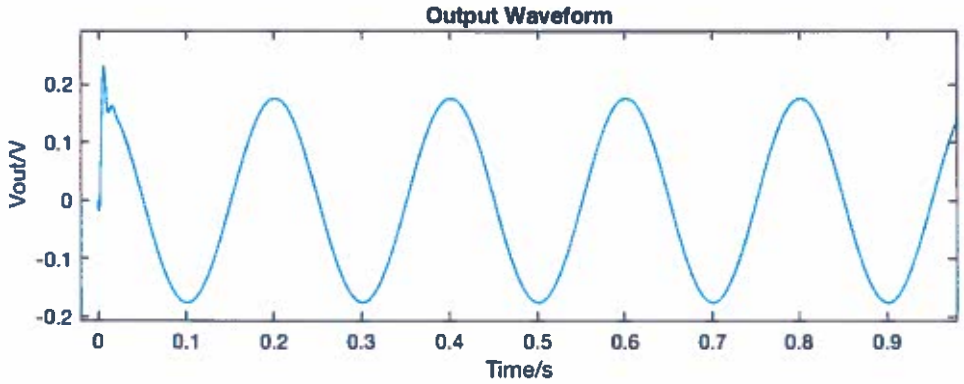
The amplitude of the output waveforms at frequency 5 HZ and 500 HZ are much smaller than the amplitude at 100 HZ. Because the RLC circuit is a bandpass filter, the centre frequency, also called the

corner frequency, is given by $\omega_c = \frac{1}{\sqrt{LC}} = 645.5\ rad/s$, and the bandwidth of the series circuit is $\Delta\omega = \frac{R_L}{L} = 466.67\ rad/s$. Then the signal outsides the frequency range [28,177] HZ will be attenuated.

For sine wave with amplitude 5 V and at frequency f=5HZ.

```
vin =@ (t) 5*sin(2*pi*5*t);                 %define input
vinn = 5*sin(2*pi*5.*ta);                   %set up an array of the input signal
```

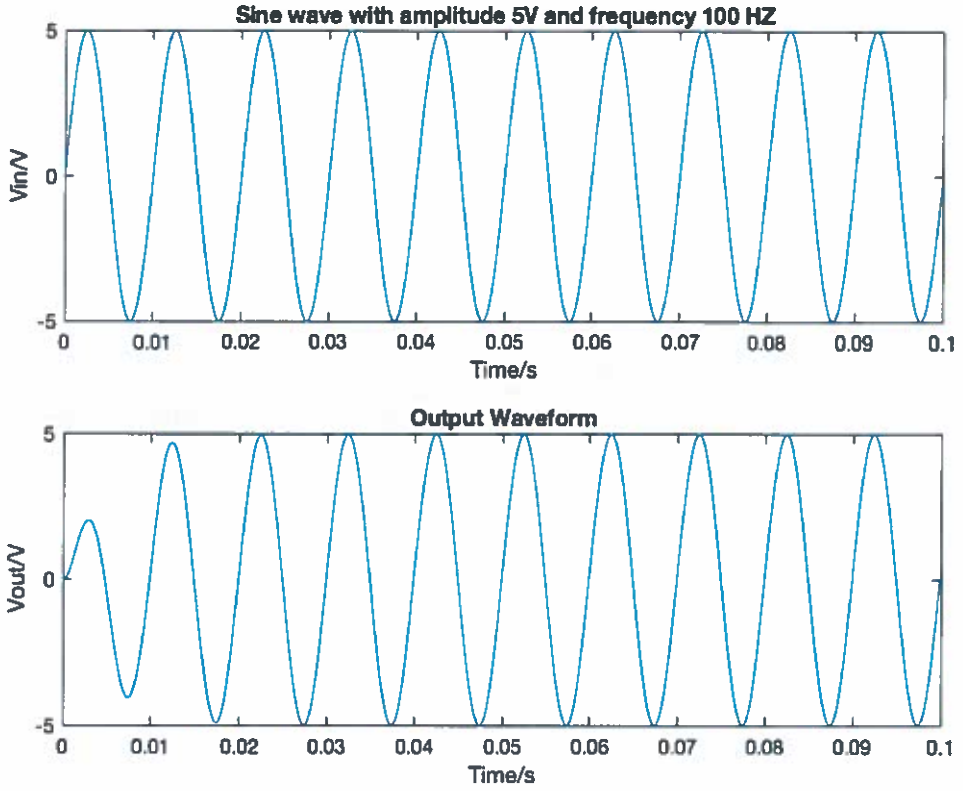




The sudden change of the sine wave at around 0.01s is caused by transient of the oscillation. The output amplitude is very small because the input signal frequency is 5 HZ, which is outside of the band-pass range.

For sine wave with amplitude 5 V and at frequency f=100HZ.

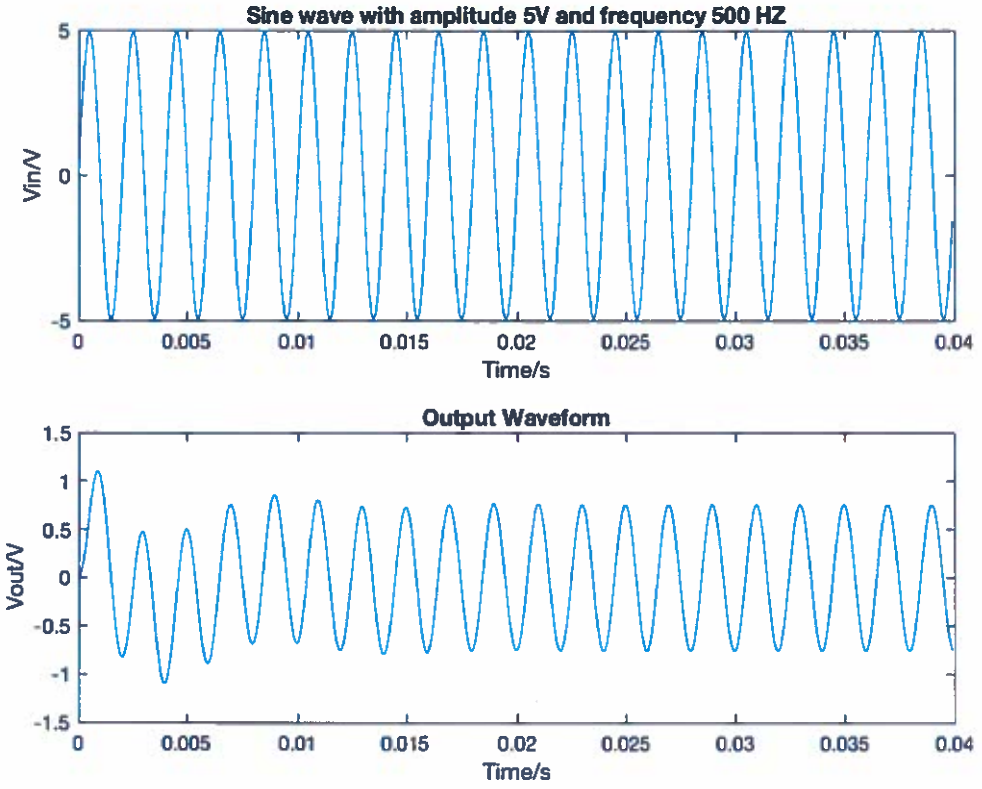
```
vin =@ (t) 5*sin(2*pi*100*t);               %define input
vinn = 5*sin(2*pi*100.*ta);                 %set up an array of the input signal
```



At frequency 100 HZ, the amplitude of the output waveform get maximized and much larger than the amplitude of the output signal at 5 HZ and 500 HZ. It is not only because the band-pass range, but also the frequency of the input signal is close to the resonant frequency of this RLC circuit, which is $\frac{645.5}{2\pi} = 102$.

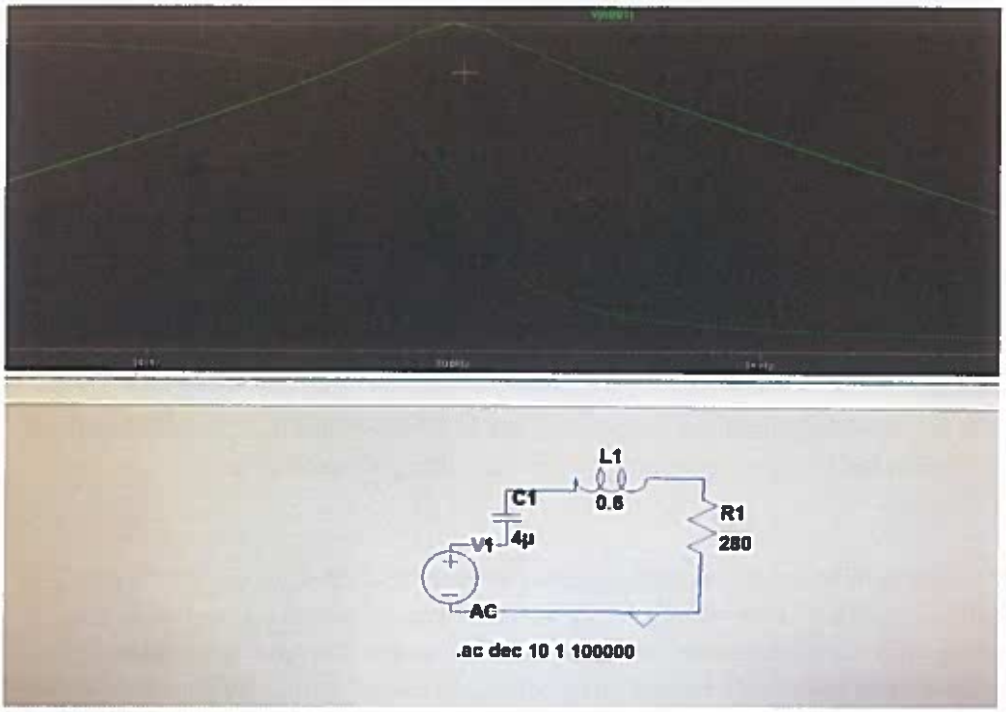
For sine wave with amplitude 5 V and at frequency f=500HZ.

```
vin =@ (t) 5*sin(2*pi*500*t);          %define input
vinn = 5*sin(2*pi*500.*ta);            %set up an array of the input signal
```



The output waveform of the input signal with frequency 500 HZ also has a sudden change caused by transient of oscillation, and the amplitude of the output waveform is very small because 500 HZ is also outside the pass-band range.

The amplitude of the output waveform with input frequency 5 HZ is smaller than that of the input frequency 500 HZ. Because from the bode plot:



The magnitude of 500 HZ is larger than that of 5 HZ.

✓ Good.

Exercise 4 – Solving the 1D Heat Equation

This 1D heat equation

$$\frac{\partial y}{\partial t} = \frac{\partial^2 y}{\partial x^2}$$

Can be solved numerically

Theory behind solution:

One can employ finite differences for both derivatives. h is the length difference of each segment, where $x = 0, h, 2h \ldots Nh = 1$ and $t = 0, k, 2k \ldots$ for as long as required.

$$\frac{\partial^2 y(x_j, t_m)}{\partial x^2} = \frac{U_{j+1}^m - 2U_j^m + U_{j+1}^m}{h^2}$$

One can generate the forward difference for the time differential by:

$$\frac{\partial y(x_j, t_m)}{\partial t} = \frac{U_j^{m+1} - U_j^m}{k}$$

By setting these two equations equal and rearranging, the following is produced

$$U_j^{m+1} = vU_{j-1}^m + (1-2v)U_j^m + vU_{j+1}^m$$

Where $v = \frac{k}{h^2}$

This equation shows that the one can calculate values for future times based on the current time.

In order to solve our problem, we will create a 2d matrix where each row will represent the heat at a different time period starting at t=0, then t=k, t=2k etc. Each element in each row will represent

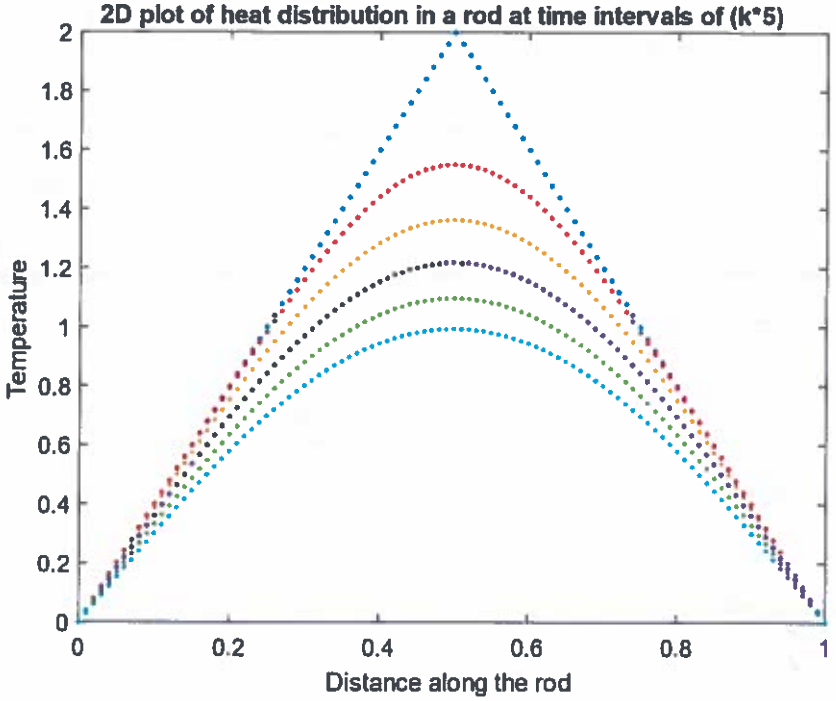
This means that the first row of our matrix corresponds to the initial conditions that we have inputted. We can also establish that the first and last column are constant boundary conditions.

We can use this formula to fill in our matrix, using a nested for loop. To do this, we use 2 for loops. The outer for loops iterates over t with intervals of k, while the inner loop iterates along the length of x, filling each successive cell in the matrix using the central equation above. The inner loop iterates from index 2 to N-1, as attempting to calculate the boundary conditions using the central algorithm will cause an out of bounds array exception.

Results:

For the line graphs, 6 lines were chosen as this was the maximum number that was perceived to not look cluttered. h was chosen to be 0.01 as this generated a sufficient amount of points to observe the graphs smoothly without being too computationally intensive. The simulation total time was chosen to be 0.2. All graphs start with the initial condition and decay down to the boundary conditions.

The Tent Function



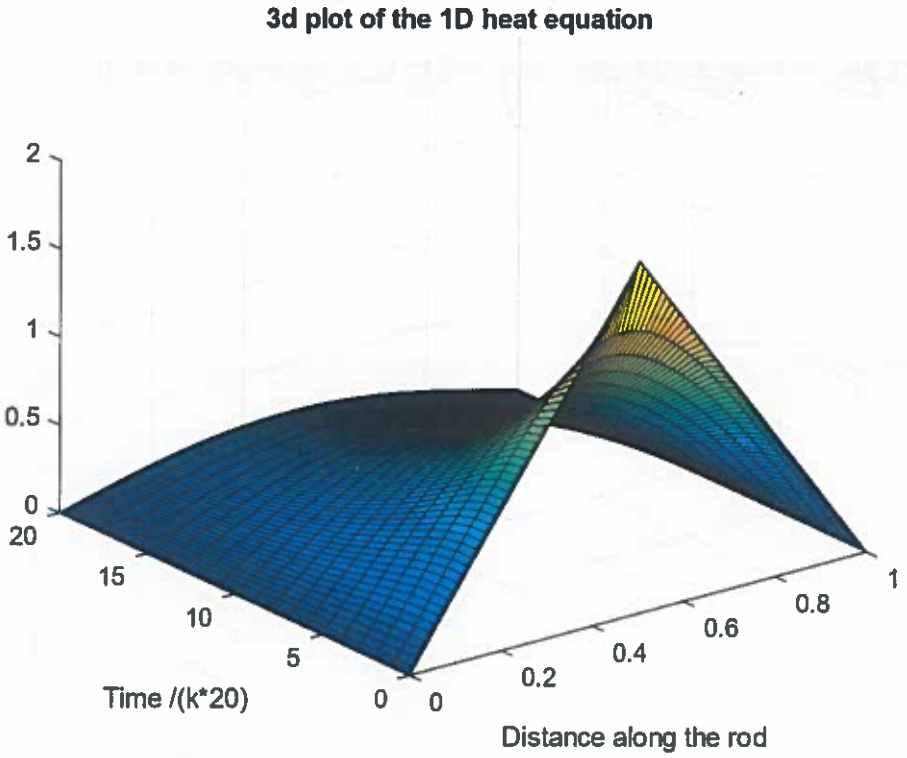
Code to Generate Plots:

```
h = 0.01; %Sets h, the distance between each points
N = (1/h)+1; % calculates N based on h. N is the number of points we are testing
xaxis = 0:h:1 % Sets up an x axis by generating an array of numbers from 0 to 1 with increments of h
xpiaxis = xaxis.*(2*pi) % Most matlab functions take a argument in radians with a period of 2pi so we
convert the x axis previously defined by multiplying each value by pi
% One can then generate the required function by calling the appropriate matlab function
IC = GetTriangle(2,0,1,100); % Fucntion GetTriangle takes the height, the start x coordinate, the end
x coordinate and the Number of points(-1)
plot(xaxis,IC,'.r'); % One can then plot these Initial Conditions to check they have been generated
correctly
% We then call the finite script previously defined. Pass in the initial conditions and the total time
of the simulation. As for the boundary conditions, we pass in the first and last value of the initial
conditions.
y = finite_script(IC,0.2,IC(1),IC(N));
surf(y);
figure();
p=[]; % One can initialise p to an empty matrix prevent previous result interference
nss = 5 % The number of times we sample y in time and thus the number of lines(-1) on our plot
for i=0:nss
   ns = 200; % One can choose to plot every ns*k values of t.
    p(i+1,:) = y(((i*ns)+1),:); % One can sample y every ns samples
end
surf(xaxis,0:nss,p); %Generates 3d plots of p
title('3d plot of the 1D heat equation');
xlabel('Distance along the rod');
ylabel(strcat('Time /(k*',num2str(nss),')'));
plot(xaxis,p','.'); % Plots the points on p as dots
title(strcat('2D plot of heat distribution in a rod at time intervals of','(k*',num2str(nss),')'))
xlabel('Distance along the rod')
ylabel('Temperature')
```

```
function y = GetTriangle(height, x0,x, dx )
%GETTRIANGLE Generates a triangular wave
%   height is the height of the wave.
% x0 is the start of the wave on the x coordinate
% x is the end of the wave on the x coordinate
% dx is the number of segments requested


dx=1/dx;
x = x0:dx:x;% Generate an array of x coordinates
height = height/2; % use the height variable to shift the saw tooth wave up the y axis
y = (sawtooth(2*pi*x,0.5)*height)+height; % Use matlab's sawtooth function to generate
the triangle wave.
end
```

↑ NOT IN FOLDER



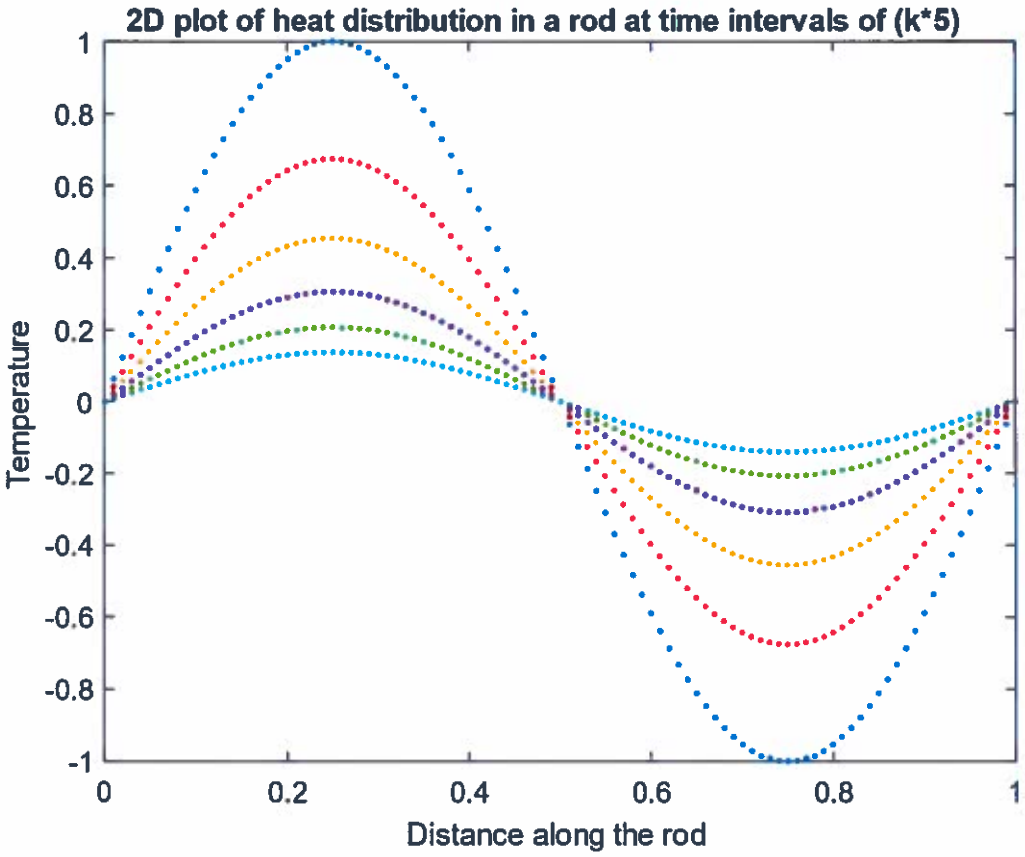
Code to Generate Plots:

```
nss=20; % The number of times we sample the result matrix y
```

Physical Interpretation:

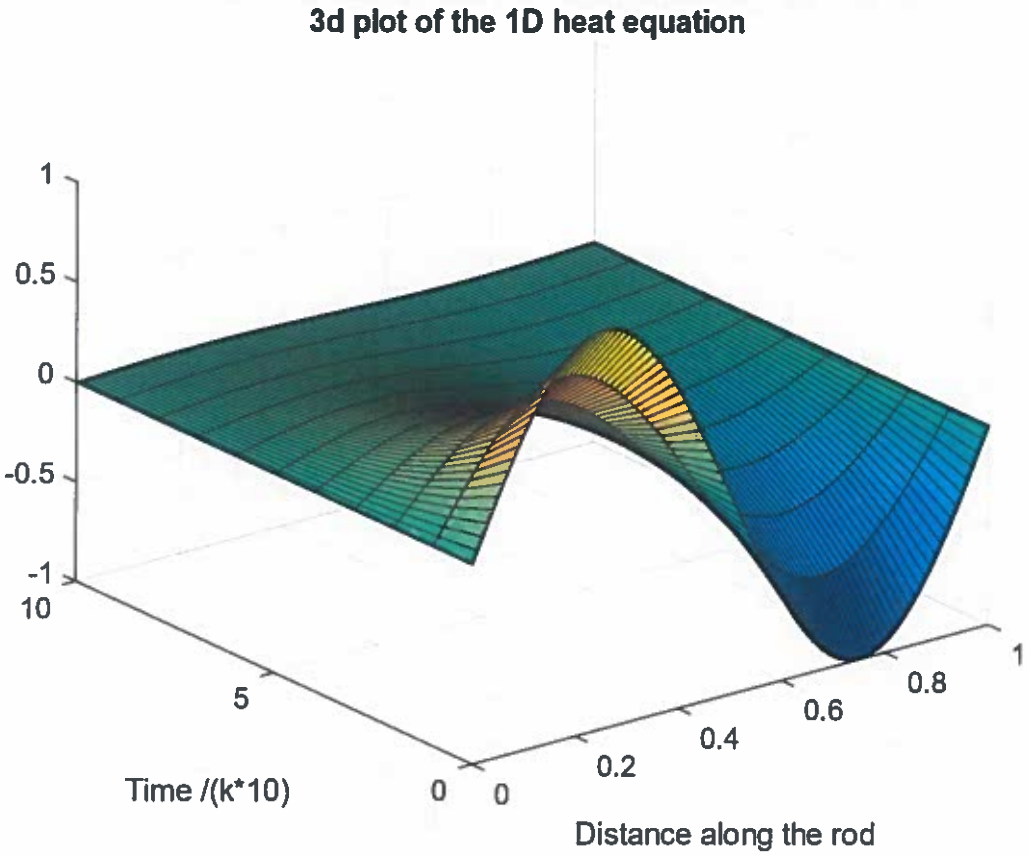
This graph represents a rod whose temperature varies as a tent function with either end kept at 0 degrees. As expected, the rod conducts heat away from the centre of the graph and siphons it off at either end. The tent function decays as the rod tends to a constant zero temperature across its whole length.

A Sine Wave Example



Code To Generate plot:

```
IC=sin(xpiaxis); % Instead of IC = GetTriangle()
```

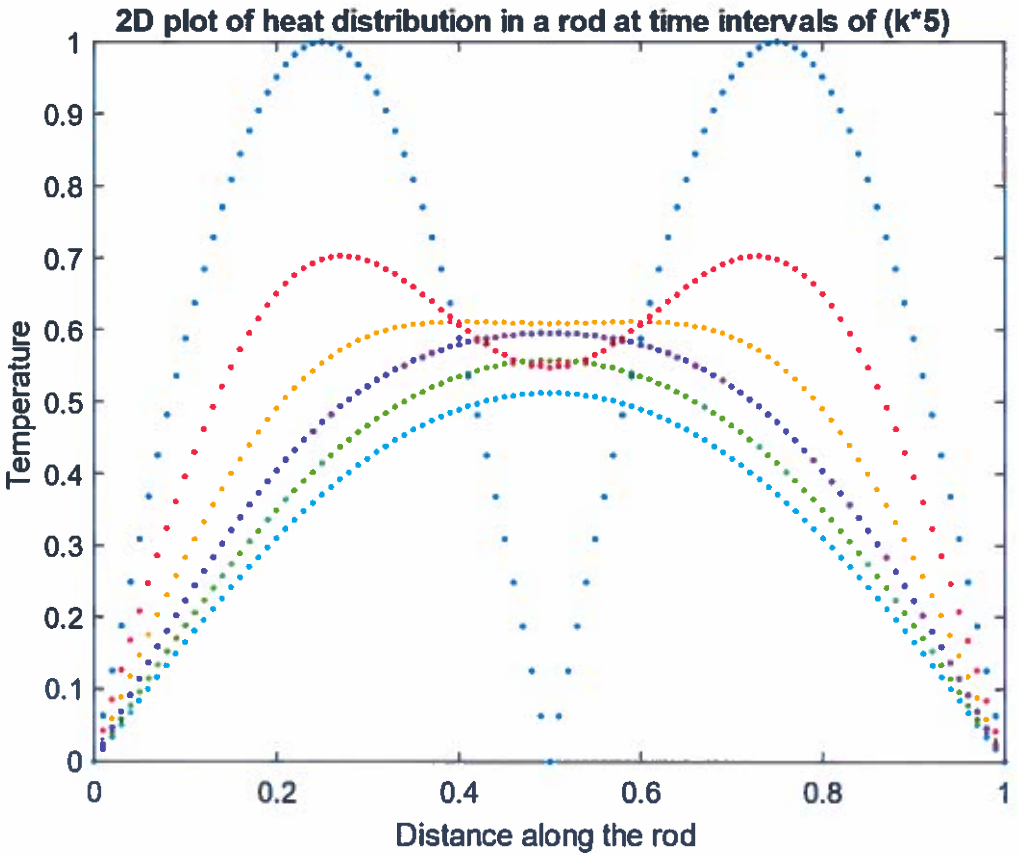


Physical Interpretation:

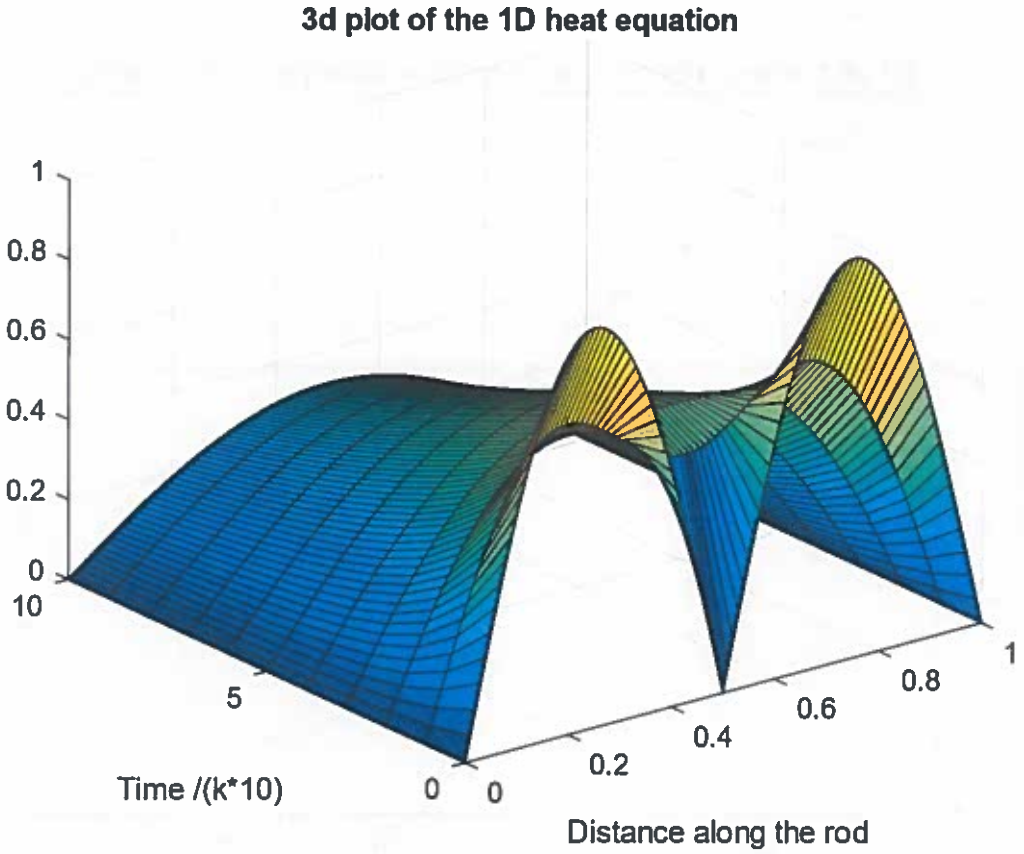
Once again, one can see that the temperature is tending towards the boundary conditions as time progresses. The difference is that whilst the left side of the graph is siphoning energy away from the rod, the right side is actually supplying energy to the rod. The symmetrical nature of the graph about x=0.5 means that the temperature in the middle is always kept at 0 degrees.

✓good

An Absolute Sinusoidal Example



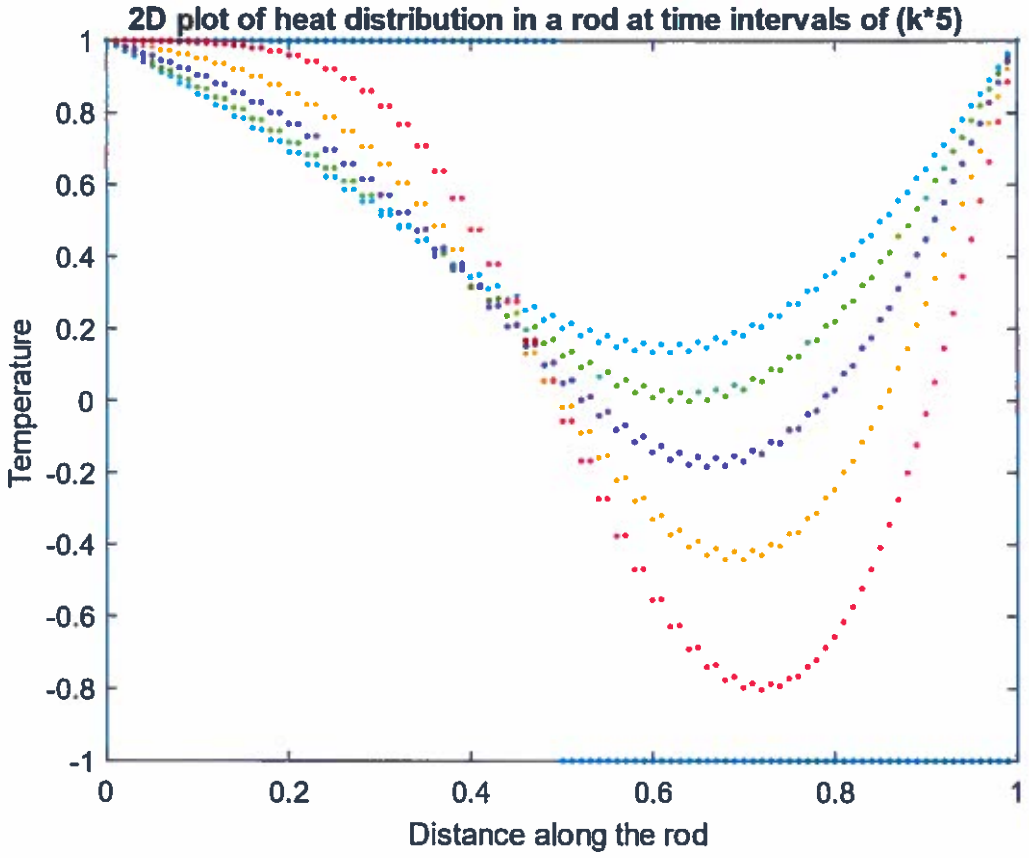

```
IC=abs(sin(xpiaxis)); % Instead of IC = GetTriangle()
```



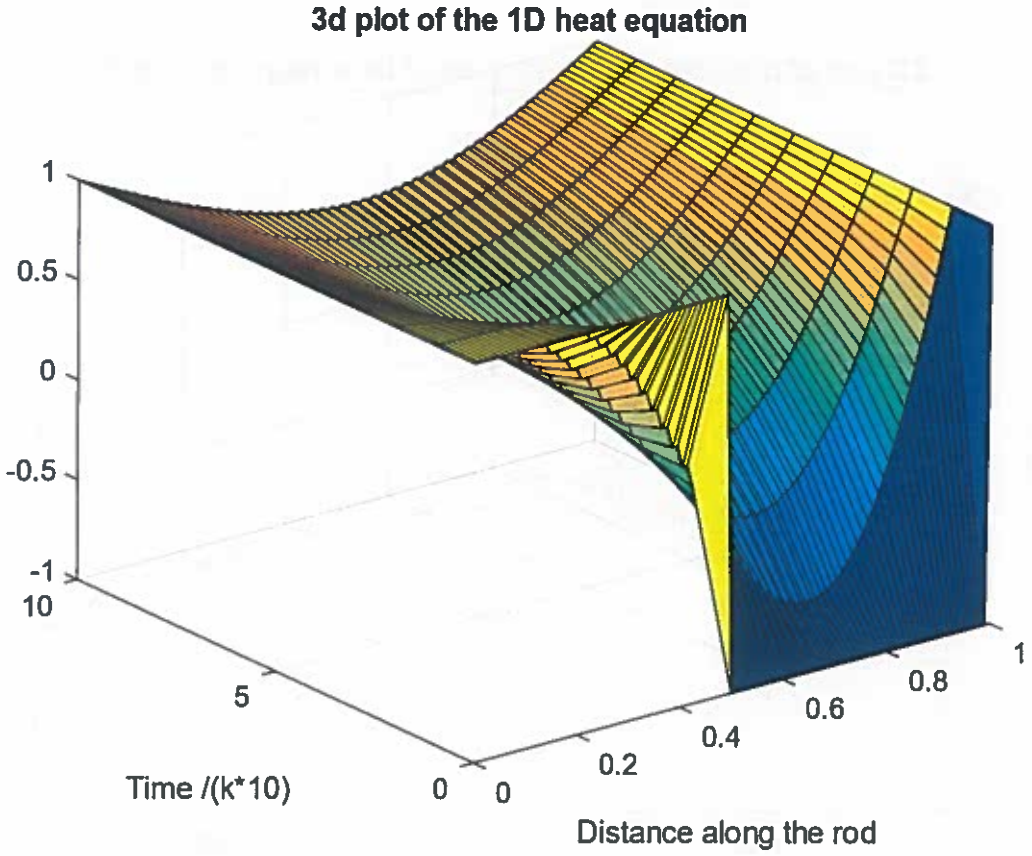
Physical Interpretation:

As seen previously, the temperature rod tends towards its 0-degree boundary conditions. However, what is interesting is that unlike the sinusoidal example, the temperature at the middle of the rod actua lly rises before it settles down to zero. This is because initially, the heat from the two peaks is conducted into the middle of the rod and thus heats it up.

A Square Wave Example



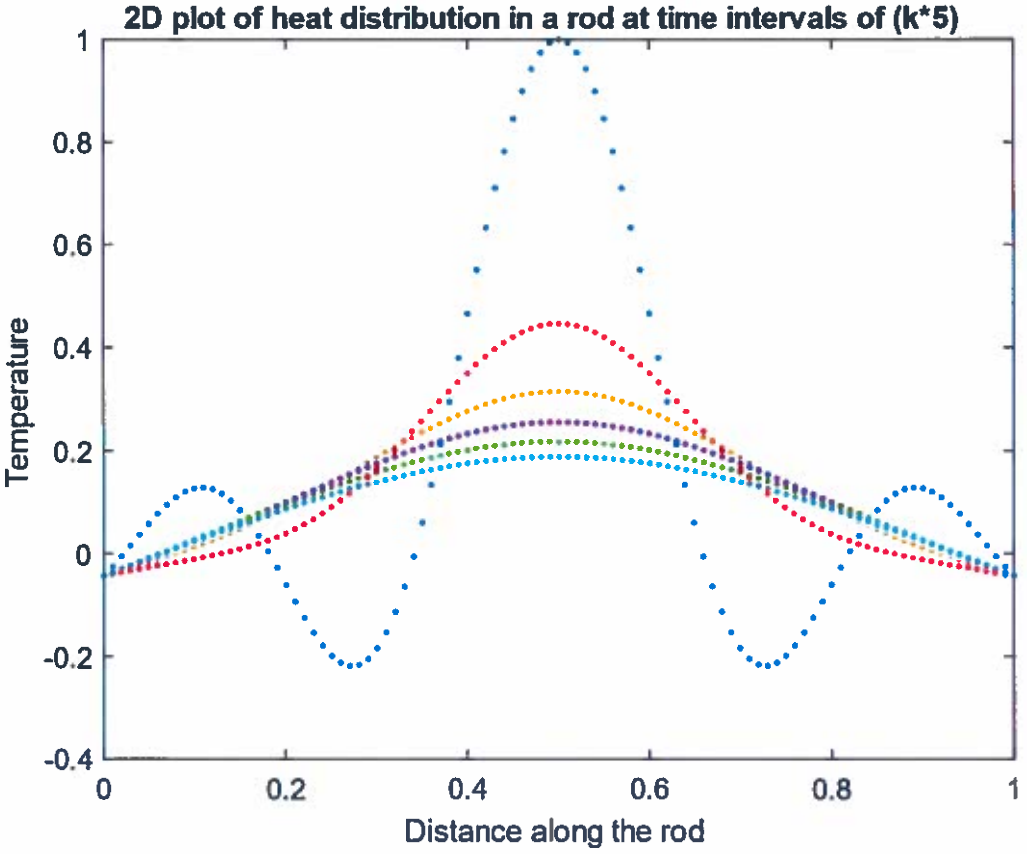

```
IC=square(xpiaxis)); % Instead of IC = GetTriangle()
```



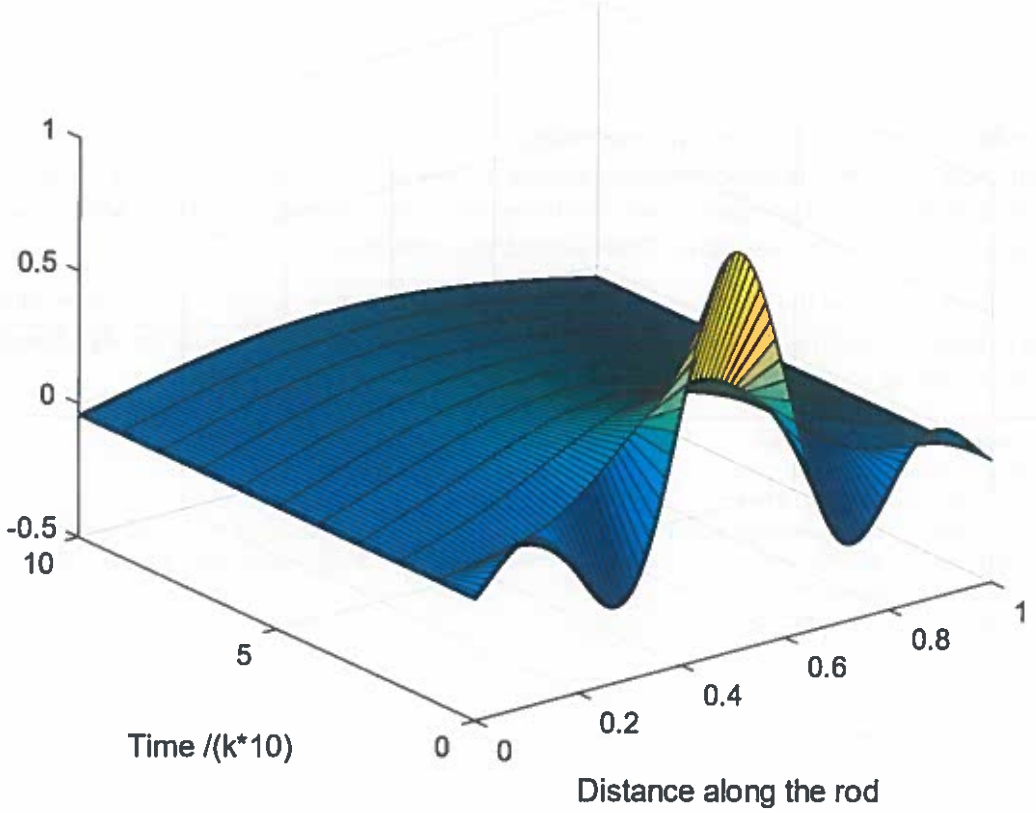
Physical Interpretation:

This is a physical situation where two bars of different temperatures and the ends are held at the temperature of the first bar. The initial rectangular shape is quickly lost due to the fact that the edges decay far more quickly than the centre. This is because there is a greater temperature differential between the sharp edges and the point after the discontinuity.

A Sampling Function (Sinc) Wave Example




```
IC=sinc(xpiaxis)); % Instead of IC = GetTriangle()
```



Physical Interpretation:

This plot is quite similar to other plots previously seen. The higher the temperature differential between two consecutive points, the faster the temperature equalises between those points. This is the reason that the central peak decays faster than the smaller peaks each side of it.

✓ v. good

Implicit Backwards Time Centred Space Method:

The issue with the Explicit Method for solving PDEs is that the solution is unstable unless v<=1/2. This limits the size of k to small numbers. If we want to simulate over a large period of time with large values of k, we can use the Implicit Backwards Time Centred Space Method.

The benefit of this is that the solution is always numerically stable. The issue is that one must solve a system of equations in order to generate each new row which is far more computationally intensive that the explicit method.

```
tridiagmatrix = ones(N,N)
  for t = 2:nt
        vec = y(t-1,2:N-1);
       tempt1 = solvetridiag(N-2,-v,(1+(2*v)),-v,vec); % At each time interval, a
  tridiagnonal matrix needs to be solved. This can be done using the solvetridiag.m
  supplied by Doctor Nucinkis
      y(t,2:N-1)=tempt1';

  end
```

This graph shows the tent function decaying with 0 degree boundary conditions. Unlike the explicit method, the result matrix does not need to be sampled at a rate of t to get a plottable matrix: it can be generated directly with a large $k = 0.1$.

where is the plot?
were is the code in the folder?

Bonus

Part (i)

When the Initial Condition doesn't match both boundary conditions, the first and last points of the given initial conditions are set to the boundary conditions. In the code, when calling finite_script rather than using the initial conditions as the boundary conditions, we can explicitly set boundary conditions.

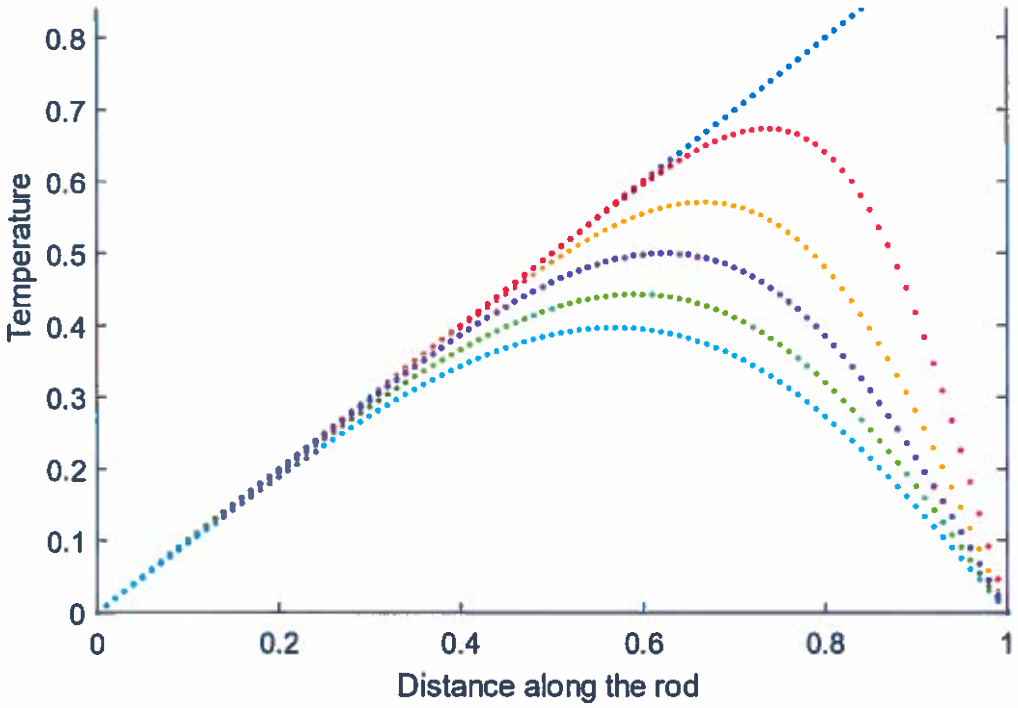
Original:

```
y = finite_script(IC,0.2,IC(1),IC(N)); % The program above uses the far most initial conditions as its boundary conditions
```

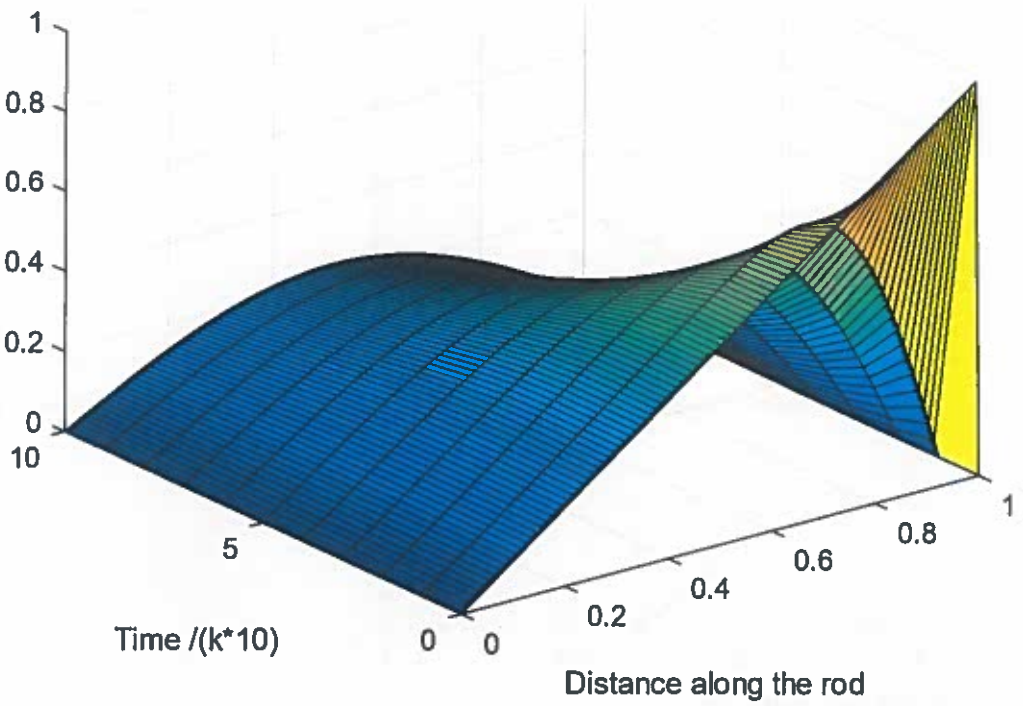
New:

```
y = finite_script(IC,0.2,0,0); % One can now see what happens when one explicitly sets the boundary conditions to 0.
```

Please excuse Issue with graph
Printer error

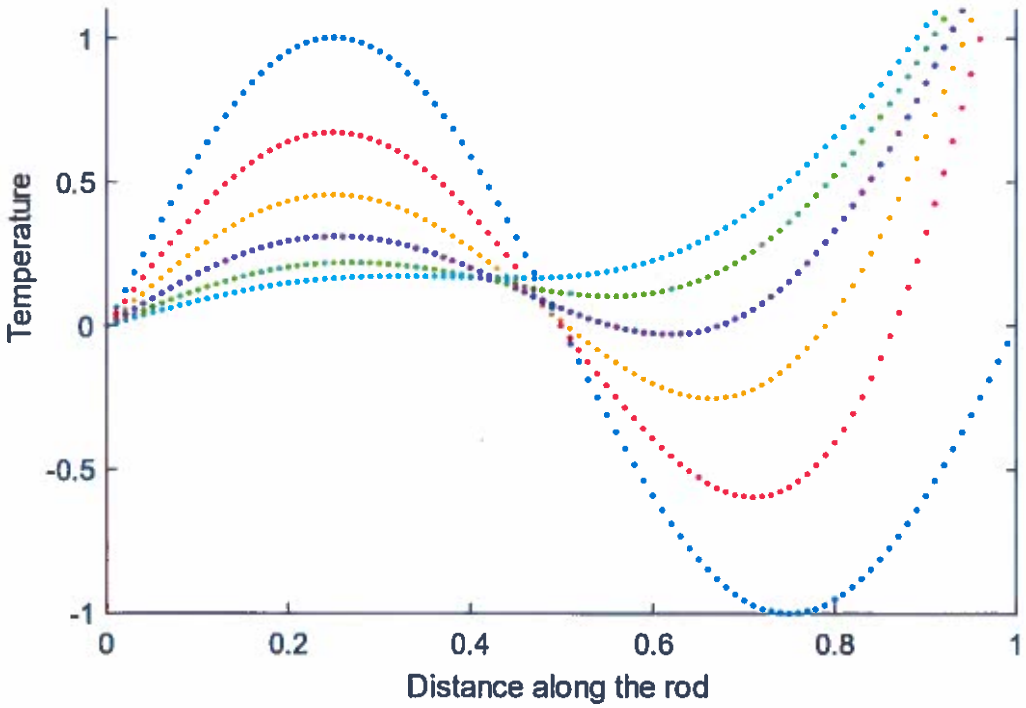


To demonstrate this one can use the graph x = y. One can observe that the boundary conditions are non-zero. When 0 degree boundary conditions are set, one can observe that the initial conditions quickly decay to the boundary conditions. The right hand side of the graph decays first due to the fact that the temperature differential between two consecutive points is much larger.

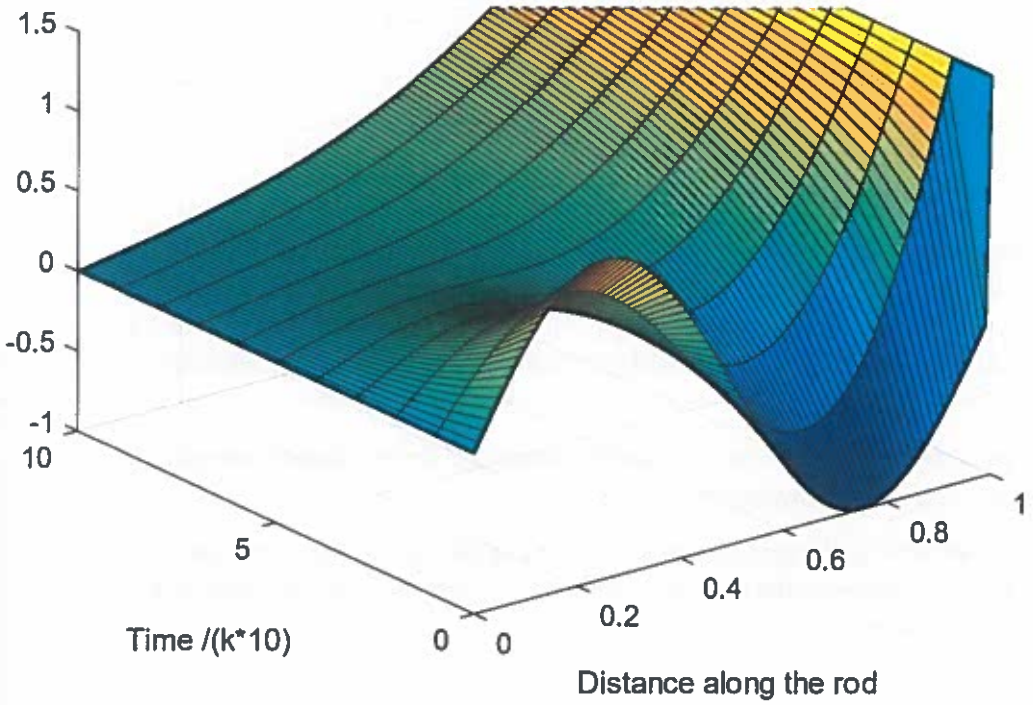


Part (ii)

This involves using constant non zero boundary conditions. In order to do this, one can set the first and last column of our solution matrix to the boundary conditions given. Here, the sine wave that was seen earlier has had boundary conditions of 0 degrees and 1.5 degrees imposed.



It can be observed that the sine wave starts to transform into the y= x. As previously observed, the further the initial condition is from the boundary conditions, the faster the initial decay rate.



By imposing the boundary conditions, a discontinuity is introduced. This is expected and can be ignored: the boundary conditions will take precedence over the initial conditions. As time progresses, the heat distribution becomes a linear interpolation between the 2 boundary conditions.

Part (iii)

This involves using time varying boundary conditions.

If this is given as a pair of functions $f_{left}(t)$ and $f_{right}(t)$, one strategy would be to sample this function at intervals of k to generate a vector of points. When initially establishing our solution matrix, the first column would be $f_{left}(t)$ and the last column would be $f_{right}(t)$. This means that when our solution is generated using the Forward Time Centred Space Method, the new limits be implanted without any further work.

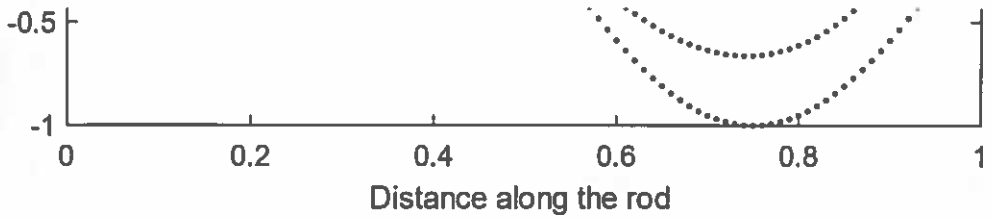
If $f_{left}(t)$ and $f_{right}(t)$ are given as a function in MATLAB, one can sample the function at intervals of k and then feed that vector in as the boundary conditions.

To do this, the finite script needs to be changed to accept b1 and b2 as functions rather than absolute values and sampling b1 and b2 to generate the array for the boundary conditions. This was done in a function called finite_script_boundary.m

Testing this script required two anonymous functions to define the boundary conditions. It was decided to test linearly increasing boundary conditions.

```
b1 = @(x) x*50;

b2 = b1;
```



In this graph, the initial conditions are the sine wave defined in dark blue at the bottom of the graph. One can observe that the boundaries are changing linearly as lime progresses as expected.

```
function temperature =  finite_script_boundary(InitialDistribution,
totalTime,b1,b2)
% Input parameters:
% xaxis: an array of x axis values for the initial distribution
% InitalConditions is the initial conditions.
% This is the Explicit method for solving the heat equation. Numerically
% unstable if v is less than 0.5.
x = InitialDistribution;
% get the number of points for each time interval
N = (size(x));
N = N(2);
%Get h where h is the interval between xi and x(i+1)
h = 1/N;
% k is the time interval between each row. K is set to half of h^2 so v<(1/2)
k = (1/2)*(h^2);
%nt is the number of rows in our matrix. It is the total number of times we
%sample the heat in the rod.
nt= ceil(totalTime/k);

% Setup the result matrix y. Each row represents a different time interval
% and each column represents the temperature at an x coordinate
y= zeros(nt,N);
%Set the first row of the matrix to be the initial conditions
y(1,:)=x;
% v is
v = k/(h*h);
b = 1-(2*v);
b1_sampled = ones(nt,1); % Pre allocate b1_smapled and b2_sampled for speed
with a set of ones
b2_sampled = ones(nt,1);

for i = 1:nt
    b1_sampled(i) = b1((i-1)*k); % Sample b1 and b2 functions at intervals of
k.
    b2_sampled(i) = b2((i-1)*k);
end

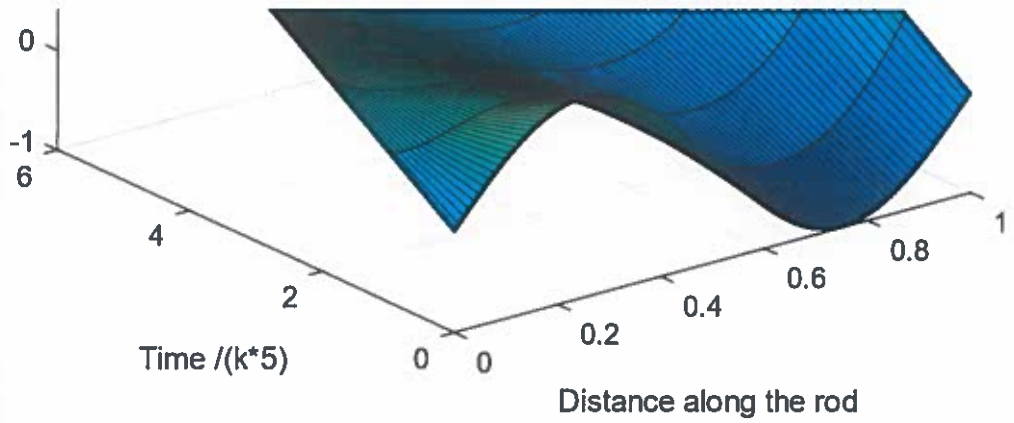
% Set the first and last columns of the matrix of the boundary conditions
y(:,1)=b1_sampled';
y(:,N)=b2_sampled';


%Implement the central algorithm
for t = 2:nt
  for j= 2:N-1

       y(t,j)=(v*y(t-1,j-1))+(b*y(t-1,j))+(v*y(t-1,j+1));

  end

end
temperature = y;
end
```



The large curve that can be seen flattens out as time progresses. However, as long as the boundary conditions continue to increase, the curve will never flatten com0letely due to the fact that the boundary conditions increase faster than the rate of heat diffusion.

✓ V. good.